ARTIFICIAL INTELLIGENCE • 23A31301T

UNIT I

Introduction and Intelligent Agents

Digital Notes | Diagrams, worked examples and exam tips

SYLLABUS COVERED IN THIS UNIT

AI problems, foundations and history of AI. Intelligent agents: agents and environments, the concept of rationality, the nature of environments, structure of agents, problem-solving agents, problem formulation.

1. What is Artificial Intelligence?

Artificial Intelligence (AI) is the branch of computer science that builds machines able to perform tasks which normally need human intelligence: understanding language, recognising images, reasoning, learning from experience and making decisions.

DEFINITION

AI is the science and engineering of making **intelligent agents**: systems that perceive their environment and take actions that maximise their chance of achieving their goals.

Different thinkers define 'intelligence' differently. Textbooks group the definitions into four approaches, shown below. Modern AI mostly follows the last one, **acting rationally**.

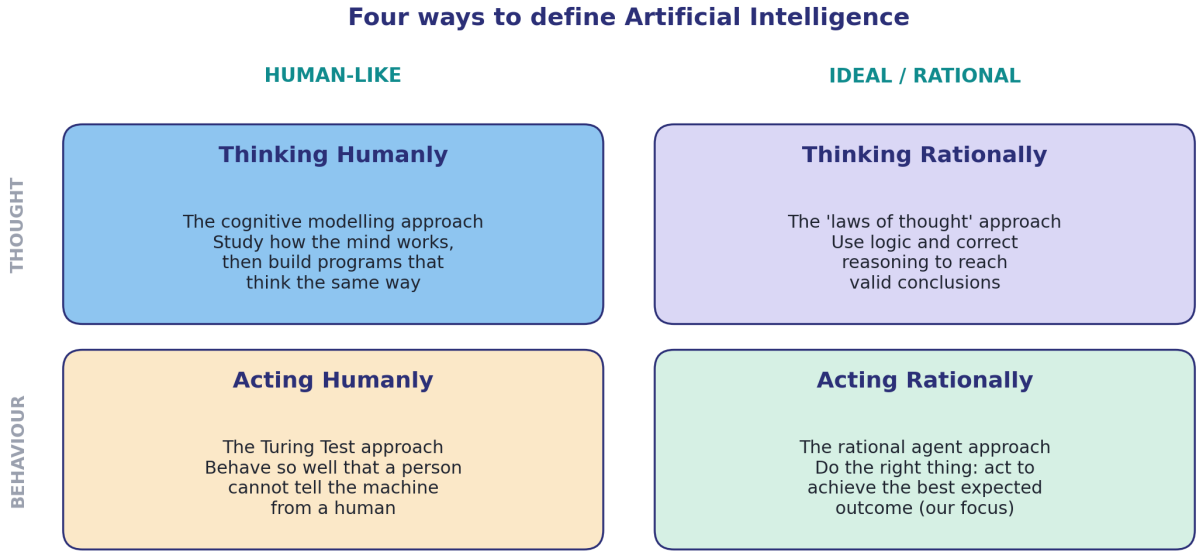


Fig 1.1 The four approaches to AI (Russell & Norvig)

Approach	Key idea	Example / test
Acting humanly	Machine behaves like a person	Turing Test (1950): a judge chats with a human and a machine; if the judge cannot tell them apart, the machine passes
Thinking humanly	Machine reasons the way people do	Cognitive science, brain imaging, models such as GPS (General Problem Solver)
Thinking rationally	Machine follows laws of logic	Aristotle's syllogisms, logic-based AI systems
Acting rationally	Machine does the 'right thing' for its goal	Rational agents: a self-driving car, a chess engine, a recommender system

EXAM TIP
The Turing Test needs these capabilities: **natural language processing, knowledge representation, automated reasoning, machine learning**. The *Total* Turing Test adds **computer vision and robotics**.

2. AI Problems

AI problems range from simple games to everyday human skills. Most are hard because the search space is huge, knowledge is incomplete and the world keeps changing.

Task area	Examples	Nature of the problem
Mundane tasks	Perception (vision, speech), natural language, common-sense reasoning, robot control	Easy for humans, surprisingly hard for machines
Formal tasks	Games (chess, Go), mathematics (theorem proving, logic)	Clear rules, but the search space is enormous
Expert tasks	Medical diagnosis, engineering design, financial analysis, scientific discovery	Need deep specialist knowledge

Typical AI problem areas

- **Natural Language Processing (NLP):** translation, chatbots, summarisation.
- **Computer vision:** face recognition, medical imaging, self-driving cars.
- **Speech recognition:** voice assistants such as Siri and Alexa.
- **Planning and robotics:** warehouse robots, route planning, drones.
- **Expert systems:** rule-based advisors in medicine and industry (Unit V).
- **Machine learning:** systems that improve with data and experience (Unit IV).

NOTE
A good way to judge an AI problem is to ask four questions: **Can it be decomposed? Can steps be undone? Is the world predictable? Does it need a good solution or just any solution?**

3. Foundations of AI

AI did not appear from nowhere. It grew out of ideas from many older fields. Each field gave AI a key tool.

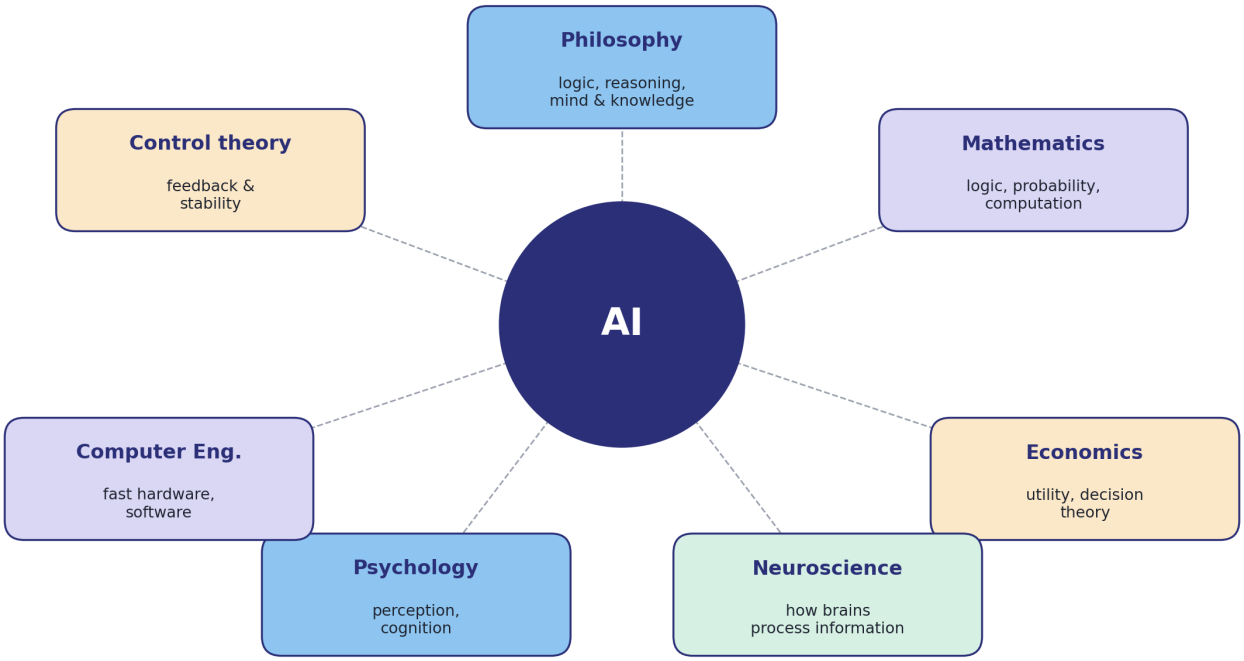


Fig 1.3 Disciplines that contribute to AI (Linguistics: language & grammar, also contributes)

Discipline	What it gave AI
Philosophy	Formal logic, theories of reasoning, the mind-body question, the idea that thought can follow rules
Mathematics	Logic, algorithms, computability, probability and statistics, optimisation
Economics	Utility theory, decision theory, game theory, Markov decision processes
Neuroscience	How neurons process information; inspiration for artificial neural networks
Psychology	Models of perception, memory, learning and problem solving
Computer engineering	Fast and cheap hardware (CPUs, GPUs), programming languages, software tools
Control theory	Feedback loops, self-regulating machines that minimise error
Linguistics	Grammar and meaning, the basis of natural language processing

4. History of AI

The story of AI is a cycle of big hopes, disappointments and fresh breakthroughs.

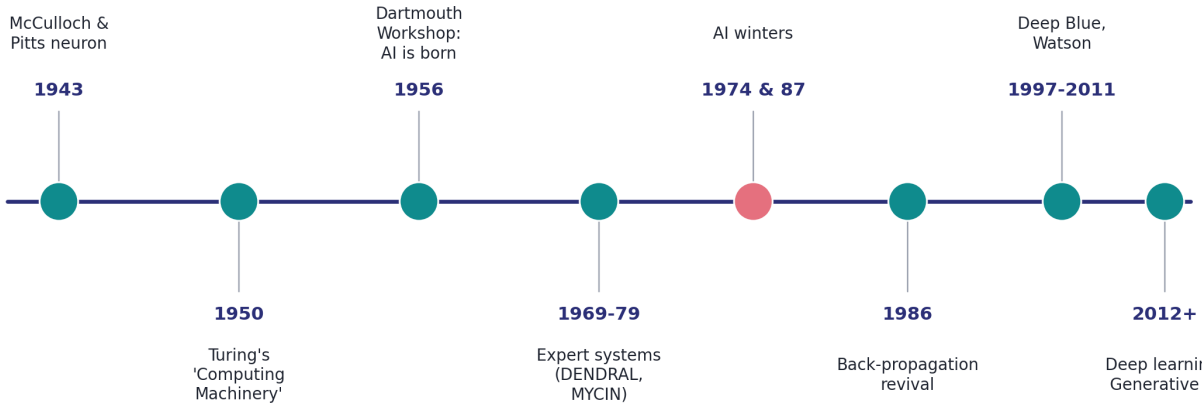


Fig 1.2 Milestones in the history of AI

Period	Phase	What happened
1943-1955	Gestation	McCulloch-Pitts artificial neuron (1943), Hebbian learning, Turing's paper (1950), first neural-network computer
1956	Birth of AI	Dartmouth Workshop; John McCarthy coins the term 'Artificial Intelligence'
1952-1969	Early enthusiasm	Logic Theorist, General Problem Solver, LISP language (1958), checkers-playing program
1966-1973	A dose of reality	Programs failed to scale (combinatorial explosion), machine translation disappointed; funding cut
1969-1986	Expert systems	DENDRAL, MYCIN, XCON: knowledge-based systems succeed commercially
1987-1993	AI winter	Expert systems proved costly to maintain; the market collapsed
1986-present	Neural nets and data	Back-propagation revival, probabilistic reasoning, machine learning, intelligent agents
1997-2016	Landmarks	Deep Blue beats Kasparov (1997), Watson wins Jeopardy! (2011), AlphaGo beats Lee Sedol (2016)
2012-today	Deep learning era	Big data and GPUs power image, speech and language models and generative AI

5. Intelligent Agents: Agents and Environments

DEFINITION
An **agent** is anything that **perceives** its environment through **sensors** and **acts** upon that environment through **actuators**.

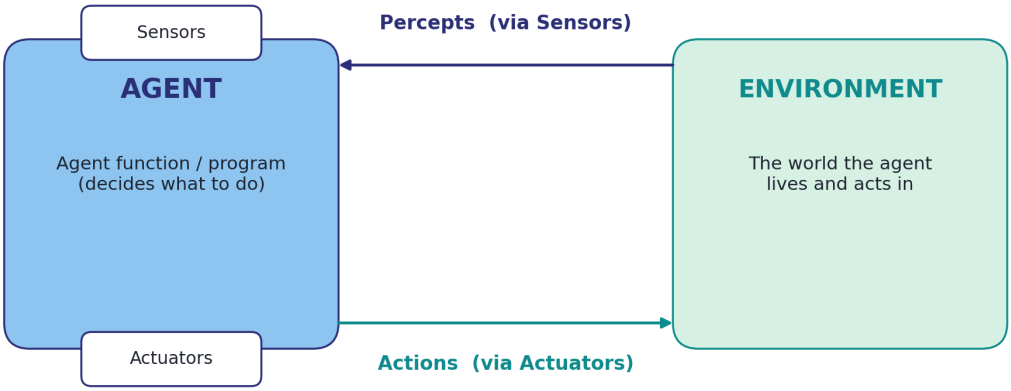


Fig 1.4 An agent perceives its environment through sensors and acts on it through actuators

Agent	Sensors	Actuators
Human	Eyes, ears, skin, nose	Hands, legs, voice
Robot	Cameras, infrared, sonar, LIDAR	Motors, wheels, grippers
Software agent	Keystrokes, files, network packets	Screen display, writing files, sending messages
Thermostat	Temperature sensor	Heater switch

Key terms

- **Percept:** the agent's input at a single moment. **Percept sequence:** the complete history of everything it has perceived.
- **Agent function:** a mathematical mapping from every possible percept sequence to an action: f : P* → A.
- **Agent program:** the concrete code that implements the agent function on a physical architecture. *Agent = Architecture + Program.*

Example: the vacuum-cleaner world. The world has two squares, A and B. The robot senses its location and whether the square is dirty. It can move Left, move Right, or Suck.

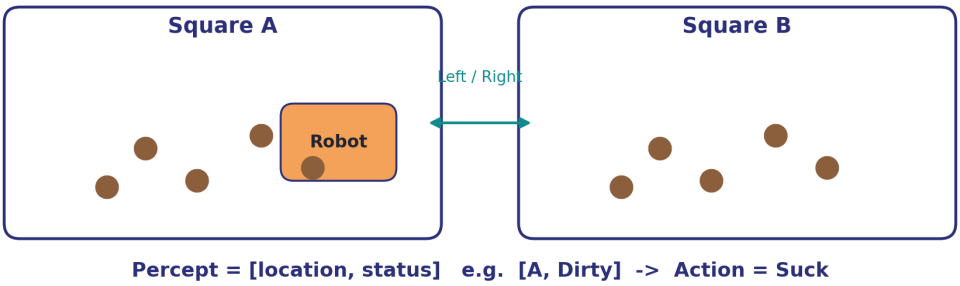


Fig 1.5 The two-square vacuum-cleaner world, a classic toy environment

Percept sequence	Action
[A, Clean]	Right
[A, Dirty]	Suck
[B, Clean]	Left
[B, Dirty]	Suck

6. The Concept of Rationality

DEFINITION
A **rational agent** is one that, for every possible percept sequence, selects an action that is **expected to maximise its performance measure**, given the evidence from the percept sequence and its built-in knowledge.

Rationality at any moment depends on four things:

- **Performance measure:** the criterion that defines success (for the vacuum robot: amount of dirt cleaned, electricity used, noise).
- **Prior knowledge:** what the agent already knows about the environment.
- **Actions available:** what the agent can actually do.
- **Percept sequence:** everything the agent has perceived so far.

REMEMBER
Rationality is **not** omniscience (knowing everything), **not** clairvoyance (predicting the future) and **not** perfection. A rational agent maximises *expected* performance, not actual performance. Example: crossing a street after looking both ways is rational even if a falling object hits you.

A rational agent should also **gather information** (exploration), **learn** from experience and be **autonomous**: depend on its own percepts rather than only on knowledge built in by the designer.

Specifying the task: PEAS description

Before designing an agent, we specify its **task environment** using **PEAS**: **P**erformance measure, **E**nvironment, **A**ctuators, **S**ensors.

Agent	Performance	Environment	Actuators	Sensors
Automated taxi	Safe, fast, legal, comfortable trip, profit	Roads, traffic, pedestrians, customers	Steering, accelerator, brake, signal, horn	Cameras, GPS, speedometer, sonar, odometer
Medical diagnosis system	Healthy patient, low cost	Patient, hospital, staff	Display of questions, tests, diagnoses	Keyboard entry of symptoms and findings
Part-picking robot	% of parts in correct bins	Conveyor belt with parts, bins	Jointed arm and hand	Camera, joint-angle sensors
Online shopping agent	Price, quality, delivery time	Websites, vendors, shippers	Follow links, fill forms, display	Web pages (HTML text)
Chess program	Win / draw / loss	Board, opponent, clock	Move pieces	Board position

7. The Nature of Environments

Environments differ along a few dimensions, and each dimension decides which kind of agent design works best.

Property	Easy version	Hard version	Meaning
Observability	Fully observable	Partially observable	Do the sensors give the complete state of the world?
Agents	Single agent	Multi-agent	Is another agent (cooperative or competitive) present?
Determinism	Deterministic	Stochastic	Is the next state fully fixed by the current state and action?
Episodes	Episodic	Sequential	Does the current decision affect future decisions?
Change	Static	Dynamic	Does the world change while the agent deliberates?
State values	Discrete	Continuous	Finite number of states and actions, or a continuous range?
Knowledge	Known	Unknown	Does the agent know the 'rules' (outcomes of actions)?

Task environment	Observable	Agents	Deterministic	Episodic	Static	Discrete
Crossword puzzle	Fully	Single	Deterministic	Sequential	Static	Discrete
Chess with a clock	Fully	Multi	Deterministic	Sequential	Semi	Discrete
Poker	Partially	Multi	Stochastic	Sequential	Static	Discrete
Taxi driving	Partially	Multi	Stochastic	Sequential	Dynamic	Continuous
Medical diagnosis	Partially	Single	Stochastic	Sequential	Dynamic	Continuous
Image analysis	Fully	Single	Deterministic	Episodic	Semi	Continuous

TIP
The **hardest** case is: partially observable, multi-agent, stochastic, sequential, dynamic, continuous and unknown. Real-world driving falls here.

8. The Structure of Agents

The **job of AI** is to design the agent program. A generic skeleton is: *receive a percept → update memory → choose an action → return it.* Five basic kinds of programs exist, in increasing order of generality.

8.1 Simple reflex agents

They choose actions using **only the current percept** and ignore history, using **condition-action (IF-THEN) rules**. Example: *IF car-in-front-is-braking THEN initiate-braking*. They are simple and fast, but work only if the environment is **fully observable**; otherwise they can loop forever.

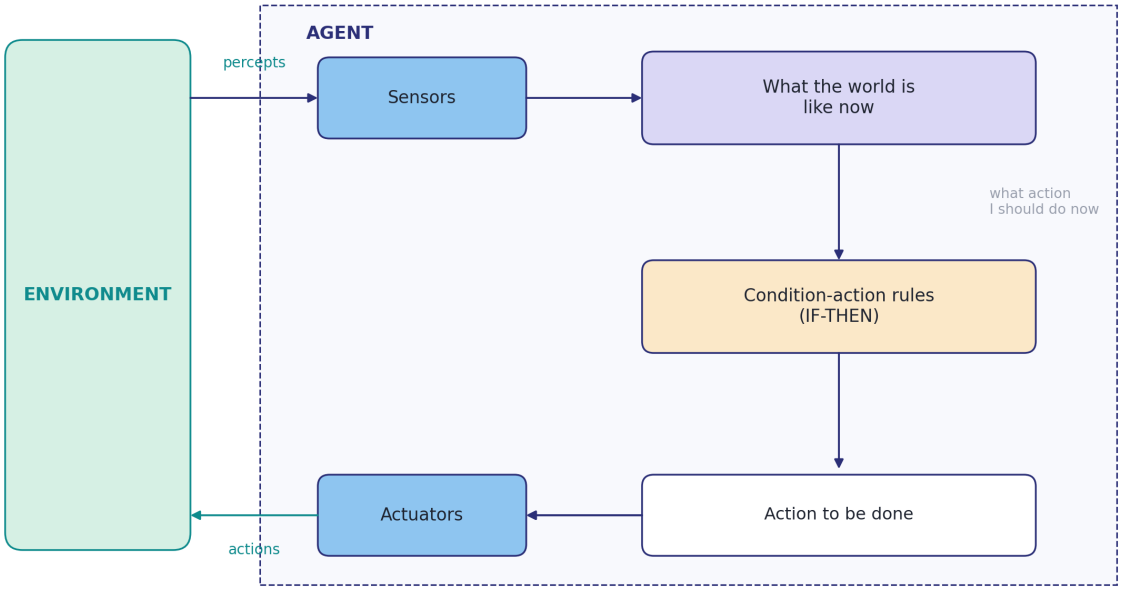


Fig 1.6 Simple reflex agent

8.2 Model-based reflex agents

To cope with **partial observability** the agent keeps an **internal state**, a model of how the world evolves and what its own actions do. It uses the model to guess the part of the world it cannot see.

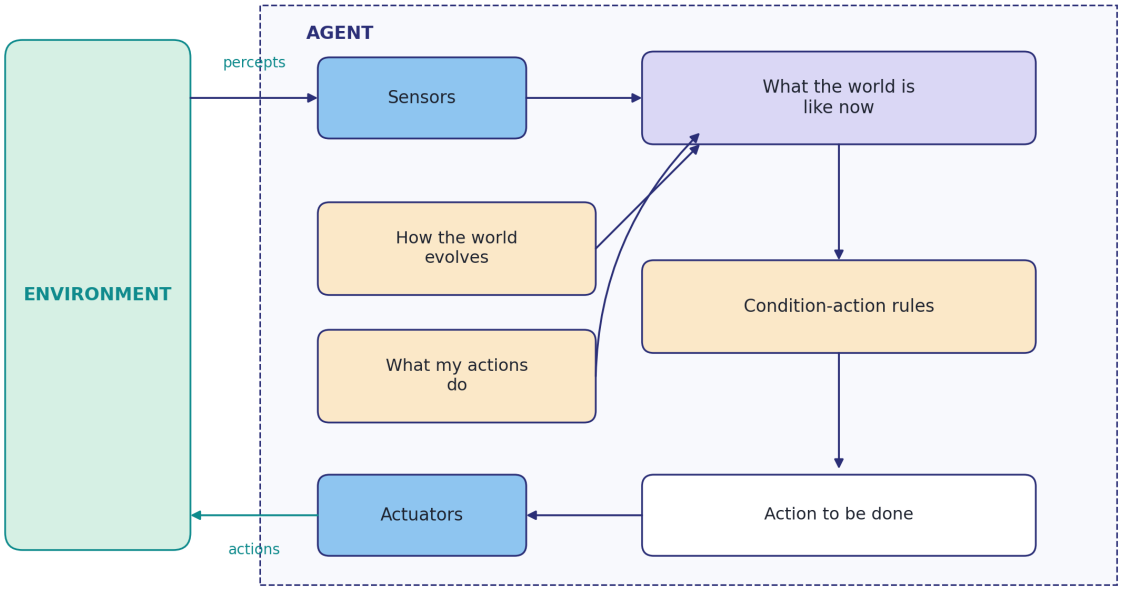


Fig 1.7 Model-based reflex agent

8.3 Goal-based agents

Knowing the current state is not enough; the agent also needs a **goal** describing a desirable situation. It considers 'What will happen if I do action A?' and chooses actions that reach the goal. This needs **search and planning**. It is more flexible: change the goal and the behaviour changes.

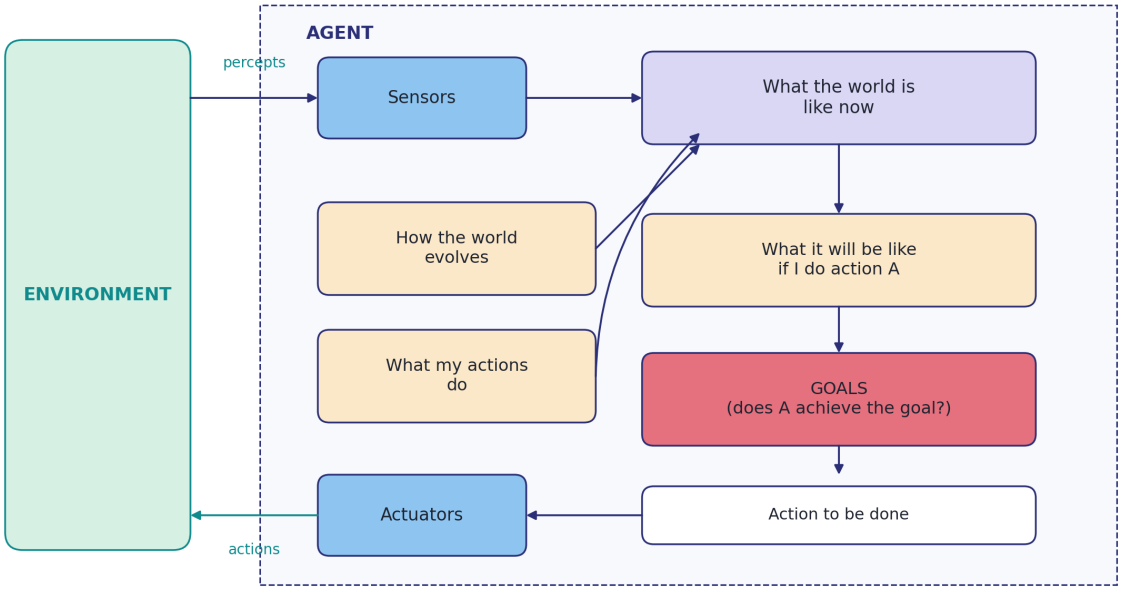


Fig 1.8 Goal-based agent

8.4 Utility-based agents

Goals give only a yes/no distinction (goal achieved or not). A **utility function** gives a numeric score of 'how happy' the agent is in a state, so it can choose the **best** route (fast *and* safe *and* cheap) and handle trade-offs and uncertainty by maximising expected utility.

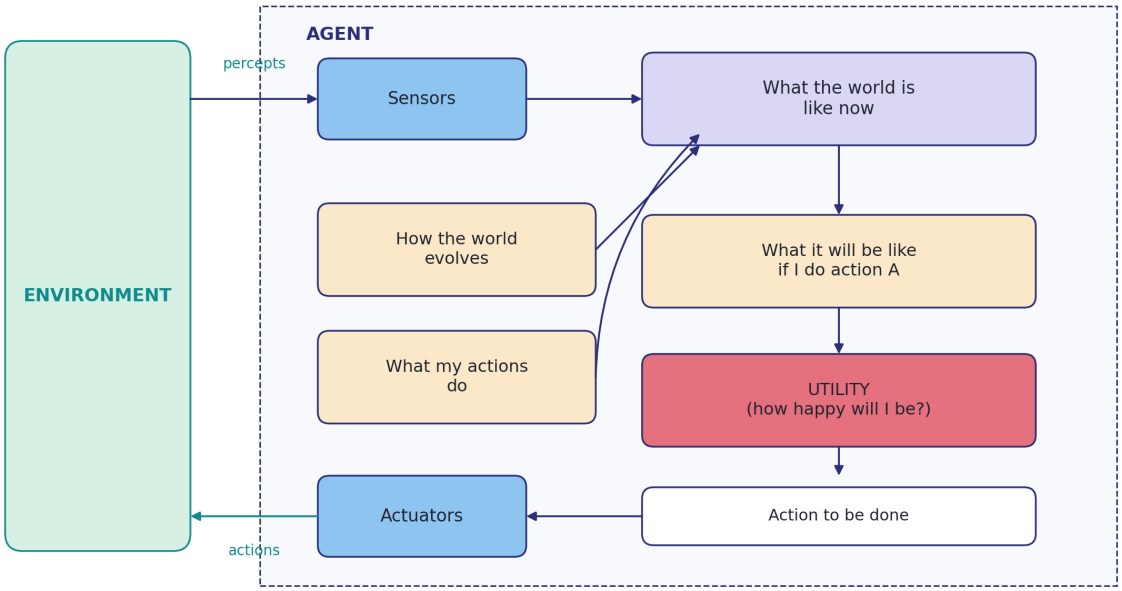


Fig 1.9 Utility-based agent

8.5 Learning agents

A learning agent can start in an unknown environment and become more competent. It has four conceptual components:

- **Performance element:** selects external actions (the agent we have described so far).
- **Learning element:** improves the performance element using feedback.
- **Critic:** tells the learning element how well the agent is doing against a fixed performance standard.
- **Problem generator:** suggests exploratory actions that lead to new, informative experiences.

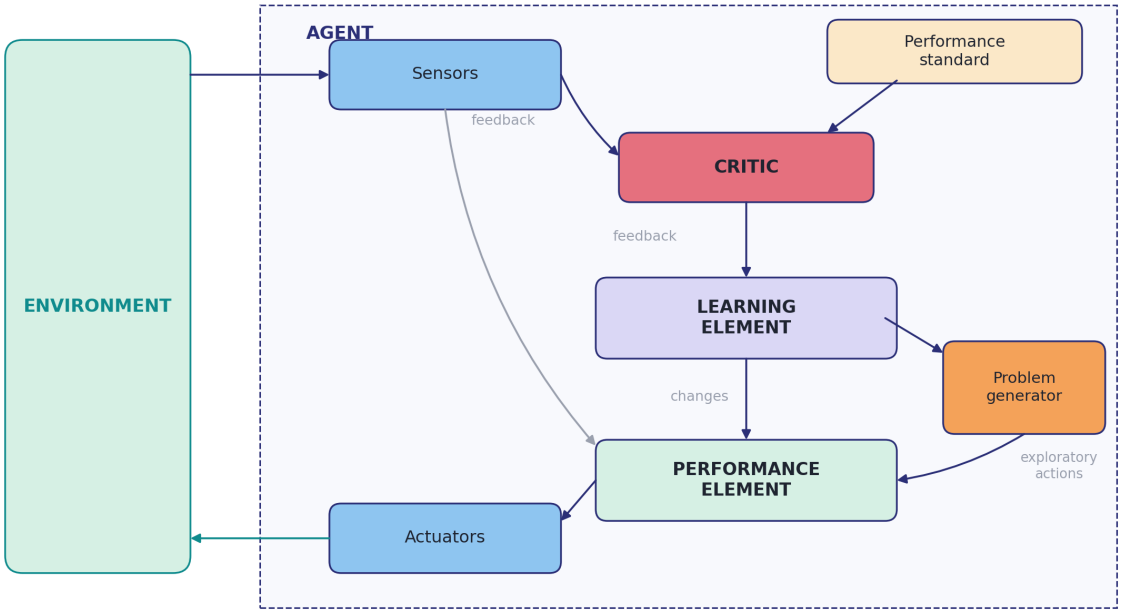


Fig 1.9b General learning agent

Agent type	Uses	Strength	Limitation
Simple reflex	Current percept + rules	Fast, simple	Fails when partially observable
Model-based	Internal state + rules	Handles hidden information	Needs an accurate world model
Goal-based	Goals + search/planning	Flexible, purposeful	Cannot compare how good goals are
Utility-based	Utility function	Best trade-offs, handles uncertainty	Utility function is hard to define
Learning	Experience and feedback	Improves over time	Needs data and time

9. Problem-Solving Agents

A **problem-solving agent** is a goal-based agent that decides what to do by **finding a sequence of actions that leads to a desirable state**. It works in four phases.

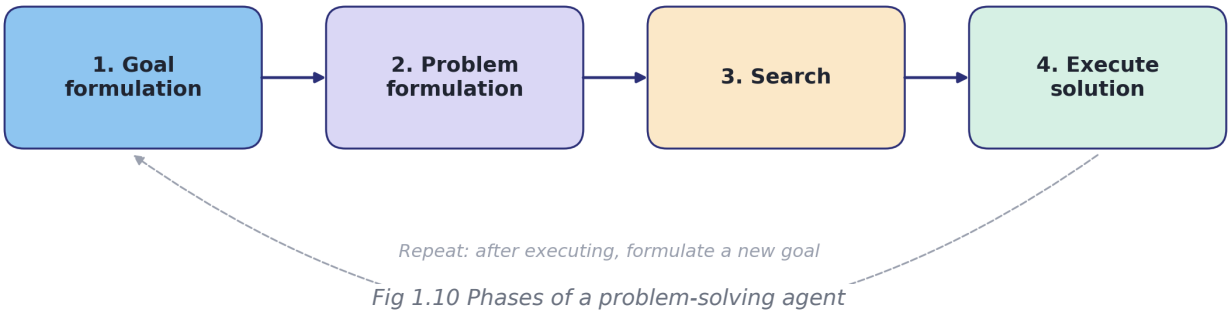


Fig 1.10 Phases of a problem-solving agent

- **Goal formulation:** decide which states count as success. This limits what the agent must consider.
- **Problem formulation:** decide which actions and states to consider (next section).
- **Search:** look for a sequence of actions that reaches the goal. The output is a **solution**.

- **Execution:** carry out the actions one by one. While executing, the agent can ignore its percepts (*open-loop* system) if the environment is observable, deterministic and known.

```
function SIMPLE-PROBLEM-SOLVING-AGENT(percept) returns an action
   state  <- UPDATE-STATE(state, percept)
   if seq is empty then
        goal    <- FORMULATE-GOAL(state)
        problem <- FORMULATE-PROBLEM(state, goal)
        seq     <- SEARCH(problem)
   action <- FIRST(seq);   seq <- REST(seq)
   return action
```

EXAMPLE
Touring Romania. An agent in Arad must reach Bucharest tomorrow. Goal: be in Bucharest. States: the cities. Actions: drive between adjacent cities. Search finds a route such as *Arad → Sibiu → Fagaras → Bucharest*; the agent then drives it.

10. Problem Formulation

Formulating a problem means defining the information the agent will use. A well-defined problem has **five components**:

Component	Description	Romania route example
1. Initial state	Where the agent starts	In(Arad)
2. Actions	Possible actions in a state, ACTIONS(s)	Go(Sibiu), Go(Timisoara), Go(Zerind)
3. Transition model	RESULT(s, a): the state reached by doing action a in state s	RESULT(In(Arad), Go(Zerind)) = In(Zerind)
4. Goal test	Checks whether a state is a goal	Is the state In(Bucharest)?
5. Path cost	A numeric cost for a path (sum of step costs c(s, a, s'))	Total distance in km

The **state space** is the set of all states reachable from the initial state. A **solution** is a path from the initial state to a goal state, and an **optimal solution** is the one with the lowest path cost.

NOTE
Abstraction matters: we remove irrelevant detail. 'Drive Arad → Zerind' hides weather, music and passengers. A good abstraction is valid (any abstract solution can be expanded into a real one) and useful (it is easier than the real problem).

Example problems

Toy problems illustrate methods and are used to compare algorithms. **Real-world problems** are those people actually care about.

(a) The 8-puzzle. A 3×3 board has eight numbered tiles and a blank. A tile adjacent to the blank can slide into it.

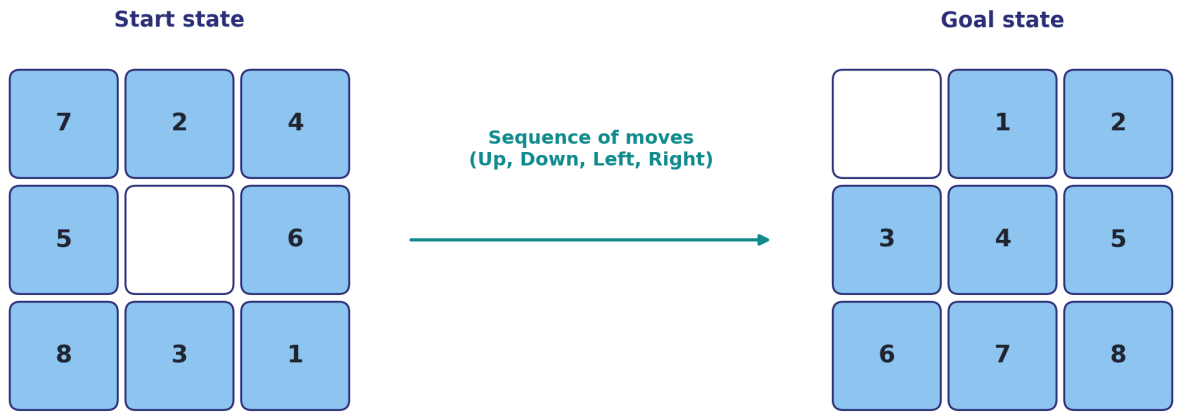


Fig 1.11 The 8-puzzle: slide tiles into the blank space to reach the goal

Component	8-puzzle
States	Location of each of the eight tiles and the blank (9!/2 = 181,440 reachable states)
Initial state	Any state (any given configuration)
Actions	Move the blank Left, Right, Up or Down
Transition model	Returns the resulting state, e.g. Left swaps the blank with the tile on its left
Goal test	State matches the goal configuration
Path cost	Each step costs 1, so cost = number of steps

(b) The vacuum world. States: agent location and dirt in each square ($2 \times 2^2 = 8$ states). Actions: Left, Right, Suck. Goal: all squares clean. Path cost: 1 per action.

(c) The 8-queens problem. Place eight queens on a chessboard so that no queen attacks another. *Incremental formulation:* start with an empty board and add one queen at a time (states = any arrangement of 0-8 queens). A *complete-state formulation* starts with all eight queens and moves them. The better version only places queens in safe squares, cutting 1.8×10^{14} states to just 2,057.

(d) Real-world problems: route finding (GPS navigation, airline travel planning), touring problems and the travelling salesperson problem (TSP), VLSI layout, robot navigation, automatic assembly sequencing and protein design.

Toy problems	Real-world problems
Concise exact description	No single agreed description
Used to compare algorithms	People actually care about the answer
Examples: 8-puzzle, vacuum world, 8-queens	Examples: route finding, TSP, VLSI layout, robot navigation

Quick Revision: Unit I at a Glance

Topic	One-line recall
AI	Building agents that act rationally to achieve goals
Turing Test	Judge cannot tell machine from human; needs NLP, knowledge, reasoning, learning
Agent	Perceives via sensors, acts via actuators; Agent = architecture + program
Rational agent	Maximises *expected* performance measure given its percept sequence and knowledge
PEAS	Performance, Environment, Actuators, Sensors
Environment types	Observable, agents, deterministic, episodic, static, discrete, known
Agent types	Simple reflex → model-based → goal-based → utility-based; learning agent improves any of them
Problem formulation	Initial state, actions, transition model, goal test, path cost

Frequently asked exam questions

- Define AI. Explain the four approaches to AI with a neat diagram.
- Explain the foundations of AI and briefly trace its history.
- What is a rational agent? Explain PEAS with at least two examples.
- Explain the different properties of task environments with examples.
- With neat diagrams, explain the five types of agent programs.
- Explain the five components of problem formulation using the 8-puzzle or vacuum world.

ARTIFICIAL INTELLIGENCE • 23A31301T

UNIT II

Searching and Game Playing

Digital Notes | Diagrams, worked examples and exam tips

SYLLABUS COVERED IN THIS UNIT

Searching for solutions, uninformed search (BFS, DFS), heuristic search (hill climbing, A*, AO*), problem reduction, adversarial search, minimax, multiplayer games, problems in game playing, alpha-beta pruning, evaluation functions.

1. Searching for Solutions

Many AI problems (route finding, puzzles, planning) are solved by **searching**: exploring possible action sequences until a goal is found. We explore the **state space** by building a **search tree**.

- **Root node:** the initial state. **Expanding** a node means applying every legal action to generate its **child nodes**.
- **Frontier (open list):** generated nodes waiting to be expanded. **Explored set (closed list):** nodes already expanded.
- **Leaf node:** a node with no children yet. The **search strategy** decides which frontier node to expand next.

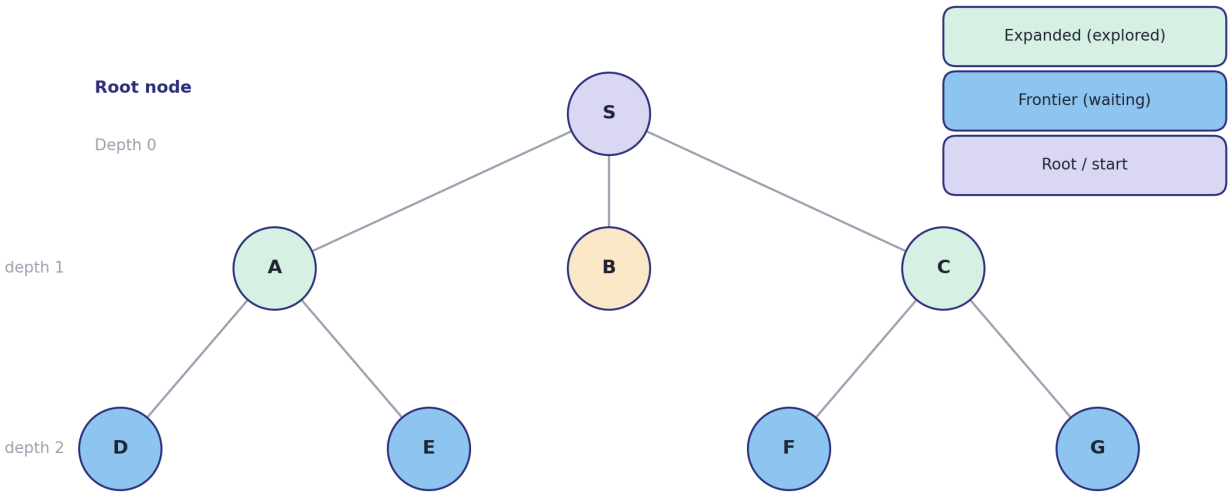


Fig 2.1 Partial search tree: expanded nodes (green), frontier (blue)

```
function TREE-SEARCH(problem) returns a solution or failure
   frontier <- { initial state of problem }
   loop do
      if frontier is empty then return failure
      node <- choose a node from frontier and remove it
      if node is a goal state then return the solution (path)
      expand node, add the resulting child nodes to frontier
```

A node in the tree stores: the **state**, the **parent** node, the **action** that created it, and the **path cost g(n)** from the root. Graph search additionally keeps an explored set to avoid repeated states (loops).

How do we compare search strategies?

Criterion	Question it answers
Completeness	If a solution exists, is the algorithm guaranteed to find it?
Optimality	Does it find the lowest-cost solution?
Time complexity	How many nodes are generated?
Space complexity	How many nodes must be stored in memory?

> **NOTE**
> Complexity is stated using three numbers: **b** = branching factor (max children per node), **d** = depth of the shallowest goal, **m** = maximum depth of the state space.

2. Uninformed (Blind) Search Strategies

Uninformed strategies have **no extra information** about how close a state is to the goal. They only know the problem definition: they can generate successors and test for a goal.

2.1 Breadth-First Search (BFS)

BFS expands the **shallowest unexpanded node** first: root, then all its children, then all grandchildren, and so on. The frontier is a **FIFO queue**.

```
BFS(problem):
   node <- (initial state, cost 0);  if goal(node) return node
   frontier <- FIFO queue with node;  explored <- empty set
   loop:
      if frontier is empty return FAILURE
      node <- POP(frontier);  add node.state to explored
      for each action in ACTIONS(node.state):
          child <- CHILD-NODE(problem, node, action)
          if child.state not in explored and not in frontier:
              if goal(child) return SOLUTION(child)
              INSERT(child, frontier)
```

2.2 Depth-First Search (DFS)

DFS expands the **deepest unexpanded node** first. It dives down one branch until a leaf is reached, then **backtracks** to the most recent node with unexplored children. The frontier is a **LIFO stack** (or recursion).

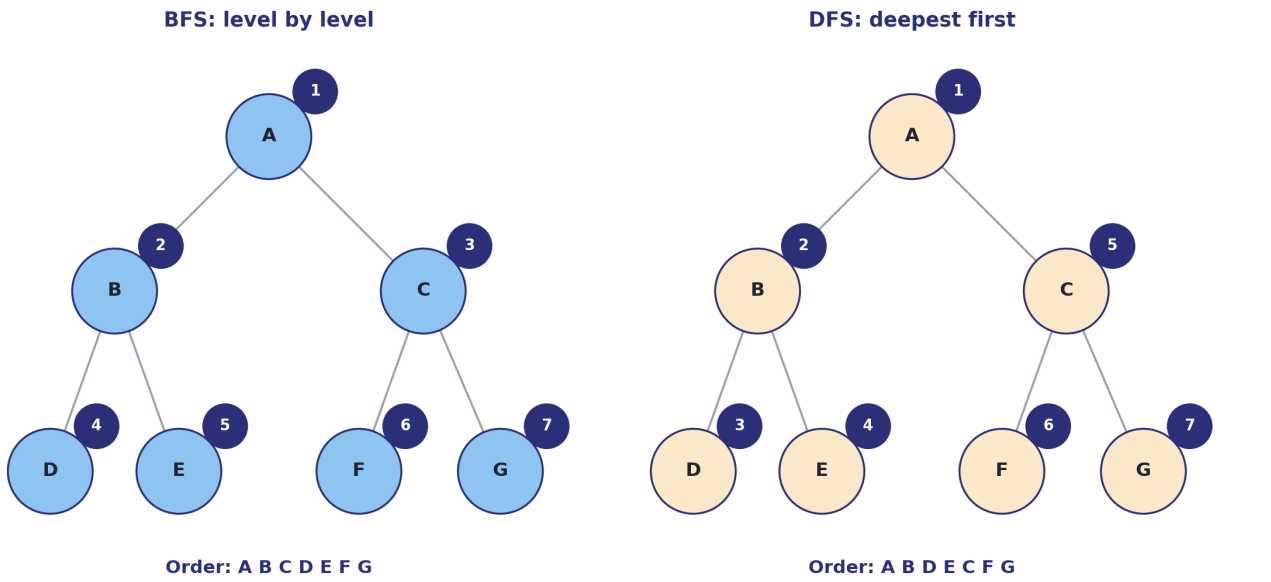


Fig 2.2 Order in which nodes are visited (small numbers) on the same tree

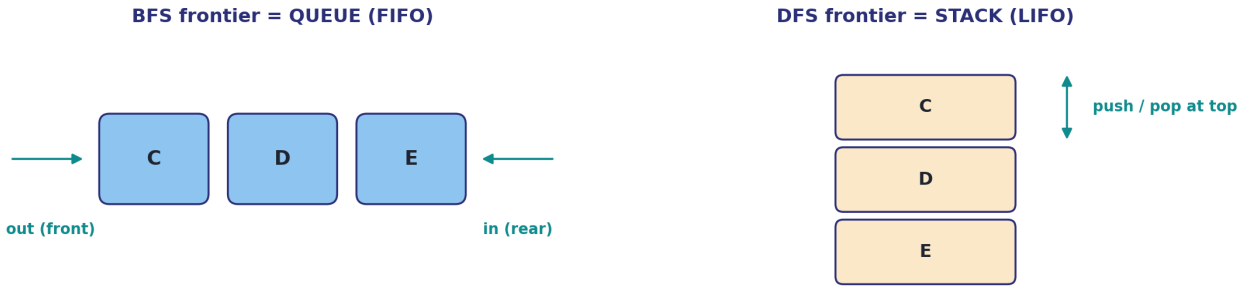


Fig 2.3 The frontier data structure decides the search order

Property	Breadth-First Search	Depth-First Search
Data structure	Queue (FIFO)	Stack (LIFO) / recursion
Complete?	Yes (if b is finite)	No (can loop or go infinitely deep); yes in finite spaces with repeat checking
Optimal?	Yes, if all step costs are equal	No
Time	$O(b^d)$	$O(b^m)$
Space	$O(b^d)$: the big weakness	O(b·m): linear, very small
Best used when	Shallow solution, need shortest path	Memory is limited, many solutions exist deep in the tree

EXAMPLE

With b = 10 and d = 6, BFS generates about 10^6 nodes and must keep them all in memory, which is the real bottleneck. DFS on the same tree with m = 6 stores only about b·m = 60 nodes.

Variants worth knowing: *Depth-limited search* runs DFS with a depth cut-off ℓ to avoid infinite paths. *Iterative deepening* runs depth-limited search with ℓ = 0, 1, 2, ... and combines BFS's completeness and optimality with DFS's low memory. *Uniform-cost search* expands the node with the lowest path cost g(n).

3. Search with Partial Information: Heuristic Search

Uninformed search wastes effort. **Informed (heuristic) search** uses problem-specific knowledge to head towards the goal. The knowledge is packaged as a **heuristic function h(n)**, an estimate of the cheapest cost from node n to a goal.

DEFINITION
Heuristic function h(n) = estimated cost of the cheapest path from n to the goal, with h(goal) = 0. It is a rule of thumb: not always right, but usually helpful.

Problem	Heuristic h(n)
8-puzzle	h_1 = number of misplaced tiles; h_2 = sum of Manhattan distances of tiles from their goal squares
Route finding	Straight-line distance from the current city to the destination
Chess	Material count (value of pieces) as a quick estimate of winning chances

Greedy best-first search expands the node that *looks* closest to the goal, using f(n) = h(n). It is fast but neither optimal nor complete.

3.1 Hill Climbing

Hill climbing is a **local search** method. It keeps only **one current state** and repeatedly moves to a neighbouring state with a better objective value. It stops when no neighbour is better, like climbing a mountain in thick fog by always stepping uphill. It uses very little memory and needs no search tree.

```
HILL-CLIMBING(problem):
   current <- MAKE-NODE(initial state)
   loop:
      neighbour <- the highest-valued successor of current
      if VALUE(neighbour) <= VALUE(current) then return current.state
      current <- neighbour
```

Variant	How it works
Simple hill climbing	Moves to the **first** neighbour that is better than the current state
Steepest-ascent hill climbing	Examines **all** neighbours and moves to the best one
Stochastic hill climbing	Chooses randomly among uphill moves (steeper moves are more likely)
Random-restart hill climbing	Runs hill climbing repeatedly from random starting states and keeps the best result

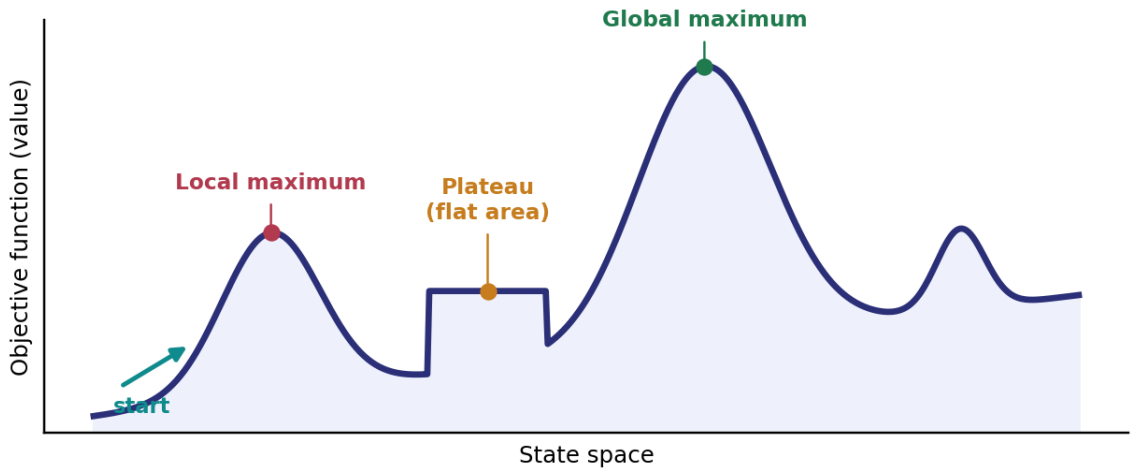


Fig 2.4 State-space landscape: where hill climbing can get stuck

Problems with hill climbing

- **Local maximum:** a peak higher than its neighbours but lower than the global maximum, so the search gets stuck.
- **Plateau:** a flat area where neighbours have equal value and there is no clear uphill direction.
- **Ridge:** a narrow slope whose peak cannot be reached by single moves, because moves go sideways instead of up.
- **Remedies:** random restarts, allowing a limited number of sideways moves, and **simulated annealing**, which occasionally accepts a downhill move to escape local peaks.

4. The A* Algorithm

A* is the most widely used informed search. It expands the node with the **lowest estimated total cost**:

> **FORMULA**
> **f(n) = g(n) + h(n)**
> g(n) = actual cost from the start to n | h(n) = estimated cost from n to the goal | f(n) = estimated cost of the cheapest solution *through* n

```
A*(start, goal):
   OPEN <- {start};  CLOSED <- {};   g(start) = 0;  f(start) = h(start)
   while OPEN is not empty:
       n <- node in OPEN with the lowest f(n)
       if n is the goal: return the path (follow parent pointers)
       move n from OPEN to CLOSED
       for each successor m of n:
           new_g <- g(n) + cost(n, m)
           if m is new, or new_g < g(m):
               g(m) <- new_g;  f(m) <- g(m) + h(m);  parent(m) <- n
               put m in OPEN (reopen it if it was in CLOSED)
   return FAILURE
```

Worked example. Find the cheapest path from S to G in the graph below (edge costs and h-values are shown).

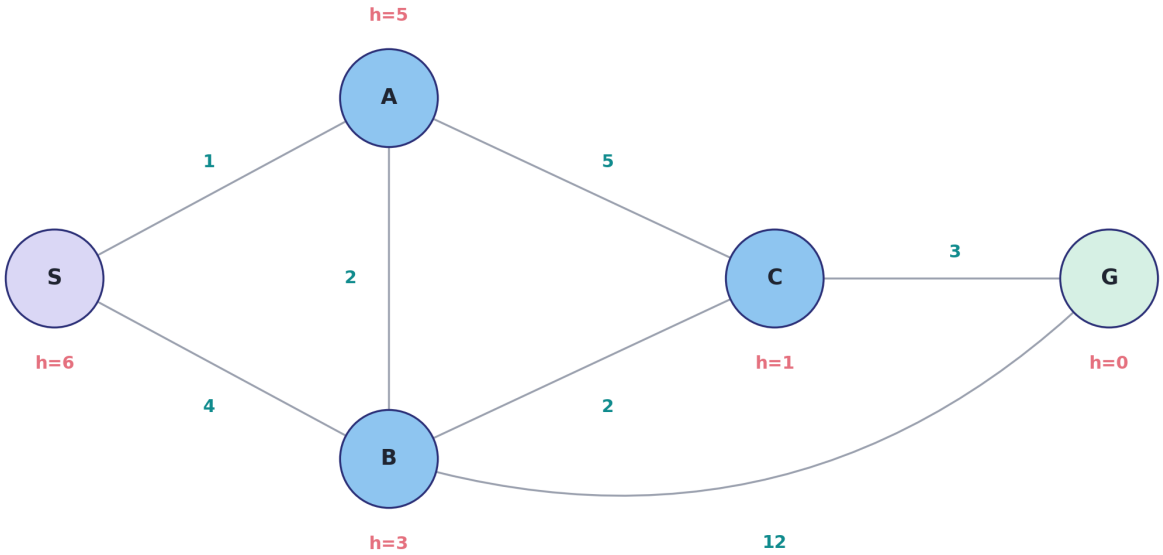


Fig 2.5 Weighted graph for A (edge labels = step cost g; red h = heuristic estimate to G)*

Step	Node expanded	Generated / updated nodes: g, f = g + h	OPEN list after the step (lowest f first)
1	S (f = 6)	A: g=1, f=6; B: g=4, f=7	A(6), B(7)
2	A (f = 6)	B improves: g=3, f=6; C: g=6, f=7	B(6), C(7)
3	B (f = 6)	C improves: g=5, f=6; G: g=15, f=15	C(6), G(15)
4	C (f = 6)	G improves: g=8, f=8	G(8)
5	G	Goal reached	Done

RESULT
Optimal path: **S → A → B → C → G** with cost 1 + 2 + 2 + 3 = **8**. Note how A* ignored the direct but expensive edge B → G (cost 12).

Properties of A*

- **Admissible heuristic:** h(n) never *over*estimates the true cost, h(n) ≤ h*(n). With an admissible h, tree-search A* is **optimal**.
- **Consistent (monotonic) heuristic:** h(n) ≤ c(n, a, n') + h(n'). Consistency implies admissibility, and f never decreases along a path.
- **Complete** (if b is finite and step costs are positive), and **optimally efficient** for a given heuristic.
- **Weakness:** keeps all generated nodes in memory, so space is the main limit, $O(b^d)$ in the worst case.

EXAM TIP
Special cases: if h(n) = 0 for all n, A* becomes **uniform-cost search**. If g(n) is ignored it becomes **greedy best-first search**.

5. Problem Reduction and the AO* Algorithm

Some problems break naturally into **independent sub-problems**. Solving the whole problem means solving *all* of them. This is **problem reduction**: the problem is repeatedly reduced to easier sub-problems, until only primitive (directly solvable) problems remain.

Such problems are represented by an **AND-OR graph** (or tree):

- **OR arcs:** alternative ways to solve a problem. *Any one* branch is enough (like ordinary search).
- **AND arcs:** sub-problems that must *all* be solved. A curved arc joins the AND branches.
- A **solution** is not a path but a **sub-graph** that solves every node reached through AND arcs.

EXAMPLE
Goal: *Get a degree*. OR-ways: (1) **buy a certificate** (single step) or (2) AND-way: **pass all semester exams AND complete the project**. Option 2 needs both parts to succeed.

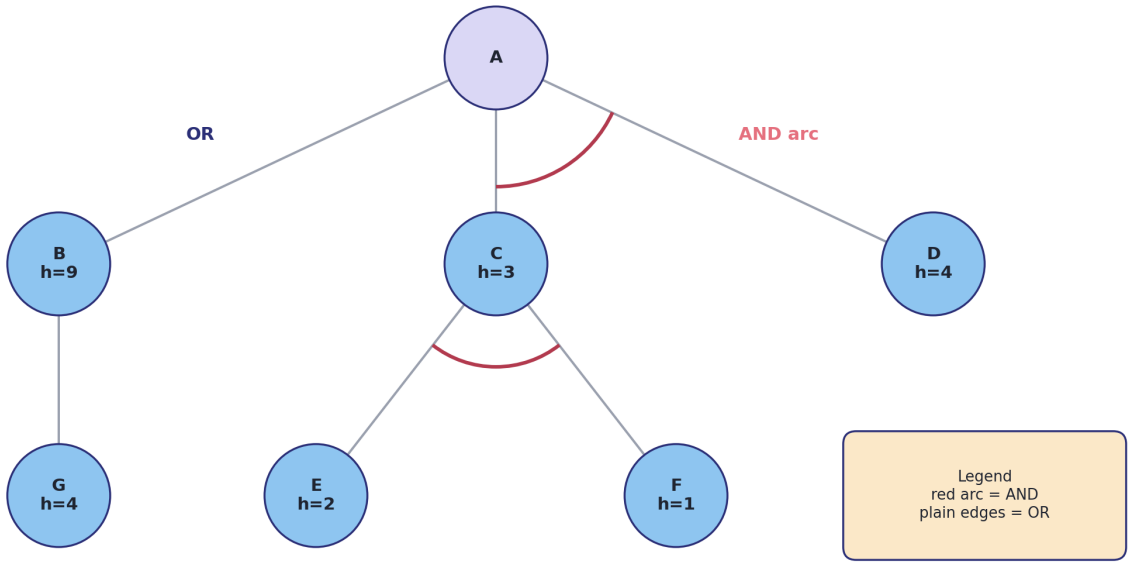


Fig 2.6 AND-OR graph for AO (A: OR of B, or AND of C and D; every edge costs 1)*

A* cannot handle AND-OR graphs, because the cost of an AND arc depends on **several** children. **AO*** is the heuristic algorithm for AND-OR graphs. For a node n and each arc to child set $\{n_1 \ldots n_k\}$, the cost of that arc is **Σ ($h(n_i)$ + cost of the edge to n_i)**. The node takes the **minimum** over its arcs.

AO* algorithm (steps)

- **1.** Start with the initial node as the current best partial solution graph; give it its heuristic value.
- **2.** Pick a non-terminal leaf node of the current **best** partial solution graph and **expand** it.
- **3.** For each new successor set, compute its cost = sum of (h + edge cost) for an AND arc, or h + edge cost for an OR arc.
- **4. Back-propagate (revise) costs** up to the root: each ancestor takes the cheapest arc. Mark the best arcs.
- **5.** Repeat 2-4 until the root is marked **SOLVED** (all nodes on the best graph are solved) or its cost exceeds a limit.

Worked example (graph of Fig 2.6)

Step	What we do	Cost computed	Best arc for A
1	Expand A: OR arc to B; AND arc {C, D}	via B = 9 + 1 = 10 via C&D = (3+1) + (4+1) = 9	AND {C, D} (9)
2	Expand C into AND {E, F}	f(C) = (2+1) + (1+1) = 5 (was 3). Revised: via C&D = (5+1) + (4+1) = 11	Now B is cheaper (10 < 11): **switch to B**
3	Expand B into G	f(B) = 4 + 1 = 5 (was 9). via B = 5 + 1 = 6	B (6): **best, solved**

Notice how AO* **revises its choice** when better information appears; this cost back-propagation is the key idea that A* does not have.

6. Game Playing: Adversarial Search

In **multi-agent competitive environments** the goals of agents conflict: one agent's gain is another's loss. **Adversarial search** decides moves in such **games** (chess, checkers, tic-tac-toe, Go). Games are interesting for AI because they are hard (huge search spaces), have clear rules and a measurable outcome.

	Deterministic	Chance involved
Perfect information	Chess, Checkers, Go, Tic-Tac-Toe	Backgammon, Monopoly
Imperfect information	Battleship, Kriegspiel	Bridge, Poker, Scrabble

A game as a search problem

Element	Meaning
S_0 (initial state)	How the game is set up at the start
PLAYER(s)	Which player has the move in state s
ACTIONS(s)	Set of legal moves in s
RESULT(s, a)	Transition model: the state after move a in state s
TERMINAL-TEST(s)	True when the game is over
UTILITY(s, p)	Final numeric value for player p at terminal state s (e.g., +1 win, 0 draw, −1 loss)

The **game tree** has states as nodes and moves as edges. We call the two players **MAX** (tries to maximise utility) and **MIN** (tries to minimise it). Each pair of moves (one by MAX and one by MIN) is called a **ply** pair; tic-tac-toe has at most 9 plies.

6.1 Minimax algorithm

Minimax computes the **optimal decision** for MAX, assuming MIN also plays perfectly. Every node gets a **minimax value**:

RULE
MINIMAX(s) = UTILITY(s) if s is terminal
= **max** of MINIMAX(RESULT(s, a)) if PLAYER(s) = MAX
= **min** of MINIMAX(RESULT(s, a)) if PLAYER(s) = MIN

```
function MINIMAX-DECISION(state) returns an action
   return argmax over a in ACTIONS(state) of MIN-VALUE(RESULT(state, a))

function MAX-VALUE(state):  if TERMINAL(state) return UTILITY(state)
   v <- -infinity;  for each a: v <- MAX(v, MIN-VALUE(RESULT(state, a)));  return v
function MIN-VALUE(state):  if TERMINAL(state) return UTILITY(state)
   v <- +infinity;  for each a: v <- MIN(v, MAX-VALUE(RESULT(state, a)));  return v
```

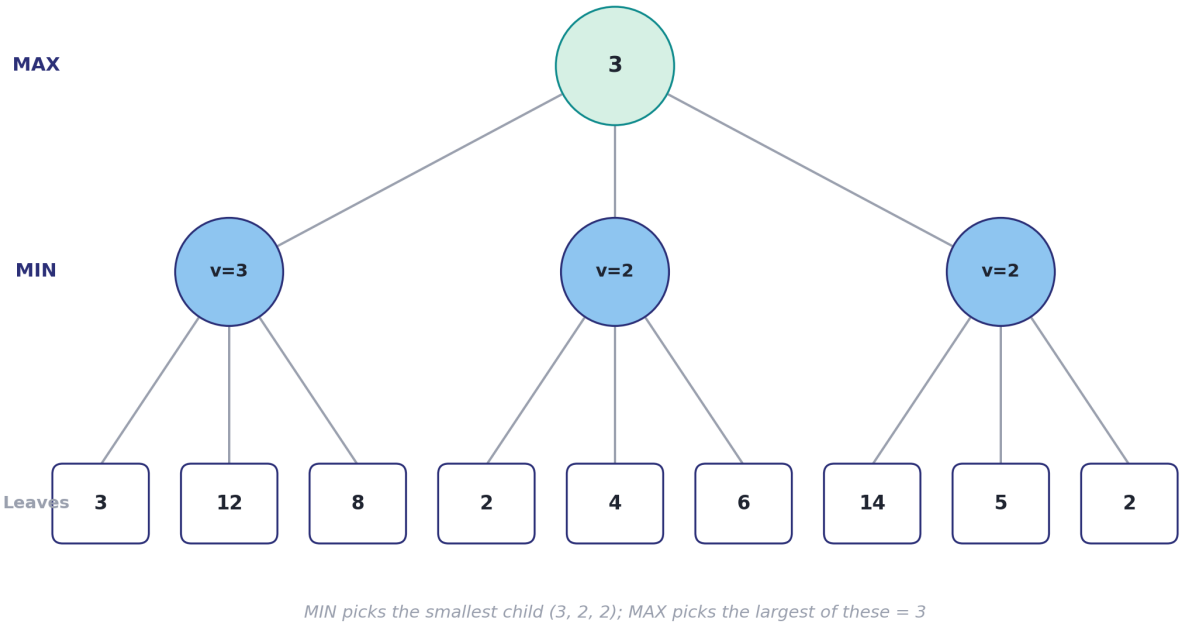


Fig 2.7 Minimax on a two-ply game tree

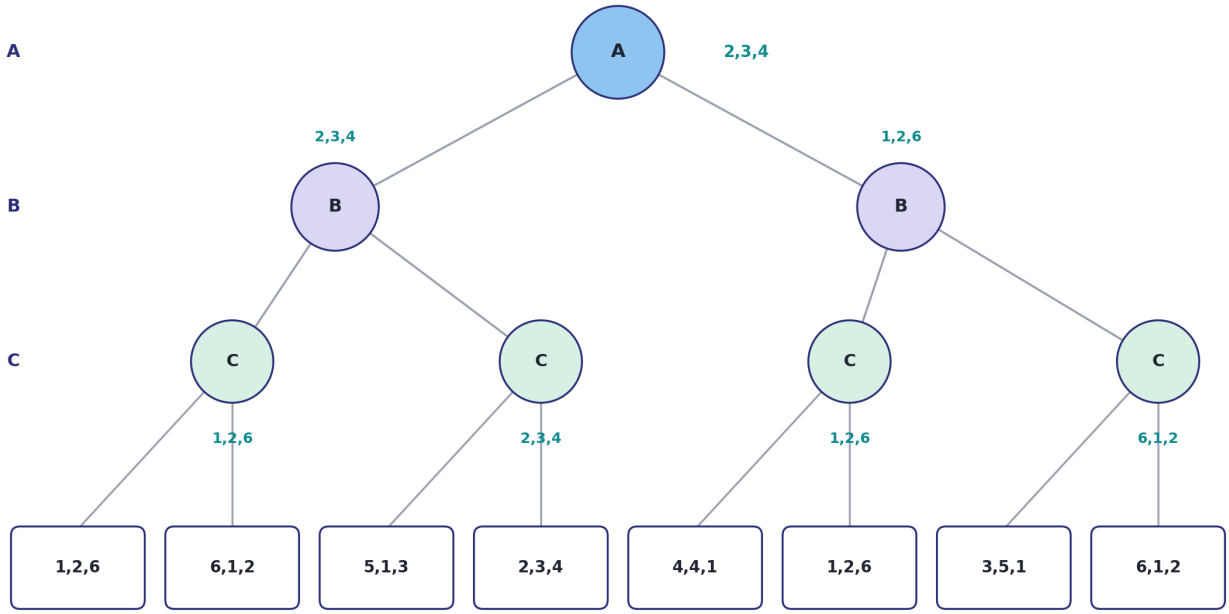
Reading the tree: the three MIN nodes pick the smallest of their leaves: min(3,12,8) = 3, min(2,4,6) = 2, min(14,5,2) = 2. The MAX root then picks the largest of {3, 2, 2} = **3**, so MAX should play the left move.

Property	Minimax
Complete	Yes, if the game tree is finite
Optimal	Yes, against an optimal opponent
Time	$O(b^m)$
Space	O(b·m) with depth-first exploration

> **NOTE**
> For chess b ≈ 35 and m ≈ 100, so b^m is about 10^{154} nodes. Exact minimax is impractical, which motivates pruning and evaluation functions.

6.2 Optimal decisions in multiplayer games

With more than two players, each node stores a **vector** of values, one utility per player. Each player, when it is their turn, chooses the move that maximises **their own component** of the vector.



Leaf = (v_A, v_B, v_C). Each player picks the child with the highest value in THEIR OWN slot.

Fig 2.9 Three-player game: each node holds a vector of utilities, one per player

- **Alliances** can form: if A and B are both weak against C, cooperating against C may be rational for both of them.
- In pure 2-player zero-sum games, the vector collapses to a single number and the vector method becomes standard minimax.
- Multiplayer games are usually **non-zero-sum**, so cooperation can emerge naturally.

7. Problems in Game Playing

- **Huge search space:** the game tree grows exponentially, so the whole tree can never be searched. We must stop at a depth limit and estimate.
- **Horizon effect:** the search stops at a fixed depth and may miss a disaster just beyond the horizon, or postpone it with pointless delaying moves.
- **Quiescence problem:** evaluating a position in the middle of an exchange (e.g., a capture) gives a misleading value. Remedy: **quiescence search**, which extends the search until the position is stable.
- **Imperfect evaluation:** the evaluation function is only a guess, and its errors can accumulate.
- **Opponent modelling and uncertainty:** the opponent may not play optimally; chance elements (dice) need **expectiminimax**.
- **Time limits:** real games have clocks, so we need **iterative deepening** and good move ordering.

8. Alpha-Beta Pruning

Minimax must examine every node. **Alpha-beta pruning** returns the **same result** as minimax but **skips branches** that cannot influence the final decision. It keeps two bounds:

- **α (alpha)** = the best (highest) value found so far for MAX along the path. It starts at $-\infty$.
- **β (beta)** = the best (lowest) value found so far for MIN along the path. It starts at $+\infty$.
- **Prune** (stop exploring a node's remaining children) whenever **α ≥ β**.

```
function ALPHA-BETA-SEARCH(state): v <- MAX-VALUE(state, -inf, +inf); return the action with value v

function MAX-VALUE(state, alpha, beta):
   if TERMINAL(state) return UTILITY(state)
   v <- -inf
   for each a in ACTIONS(state):
        v <- MAX(v, MIN-VALUE(RESULT(state, a), alpha, beta))
        if v >= beta then return v          # beta cut-off
        alpha <- MAX(alpha, v)
   return v
function MIN-VALUE(state, alpha, beta):     # symmetric
   v <- +inf
   for each a in ACTIONS(state):
        v <- MIN(v, MAX-VALUE(RESULT(state, a), alpha, beta))
        if v <= alpha then return v         # alpha cut-off
        beta <- MIN(beta, v)
   return v
```

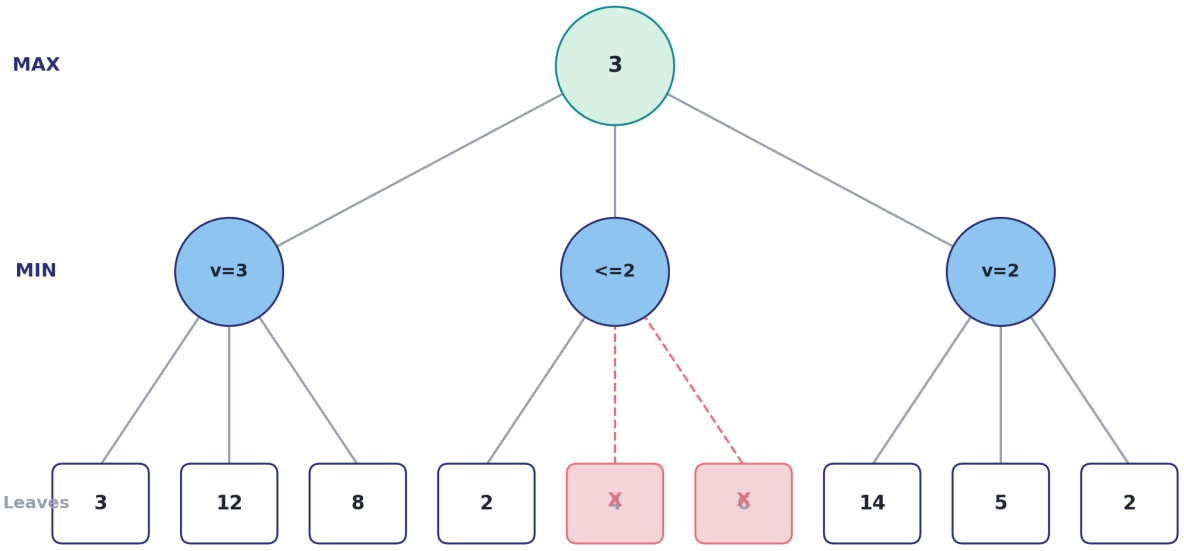


Fig 2.8 Alpha-beta pruning on the same tree

Trace: At the left MIN node, v becomes 3, so α = 3 at the root. At the middle MIN node the first leaf is 2, so v ≤ 2 ≤ α = 3: MAX will never choose this branch, so the leaves **4 and 6 are pruned**. The right node is searched (14, 5, 2) and gives 2. The root value is still **3**, the same as minimax, with fewer nodes examined.

Aspect	Details
Correctness	Pruning never changes the final minimax value or the best move
Best case (perfect move ordering)	Time $O(b^{m/2})$: searches **twice as deep** in the same time. Branching factor effectively becomes √b
Worst case	No pruning, $O(b^m)$, same as minimax
Move ordering	Try the best moves first (e.g., captures first in chess) to prune more; use iterative deepening to learn good orderings

Evaluation functions

Because we cannot search to the end of the game, we **cut off** the search at some depth and apply a **heuristic evaluation function EVAL(s)** that estimates the expected utility of the state, as if it were a terminal state.

FORMULA
EVAL(s) = $w_1f_1(s) + w_2f_2(s) + \ldots + w_nf_n(s)$ (weighted linear function of features)

- **Features f_i** are measurable properties of the position; **weights w_i** show their importance (these can be learned).
- **Chess example:** f_1 = (white pawns − black pawns) with weight 1, f_2 = (white knights − black knights) with weight 3, bishops 3, rooks 5, queens 9.
- **Requirements of a good EVAL:** it must agree with the true utility at terminal states, be **fast to compute**, and correlate strongly with the actual chance of winning.
- **Cut-off test:** replaces the terminal test; for example, stop at depth d, or when the position is quiet (quiescence).

Quick Revision: Unit II at a Glance

Topic	One-line recall
BFS / DFS	Queue, shallow-first, complete and optimal (equal costs) / stack, deep-first, low memory, not optimal
Heuristic h(n)	Estimate of the remaining cost to the goal; admissible = never overestimates
Hill climbing	Always move uphill; may get stuck at a local maximum, plateau or ridge
A*	f(n) = g(n) + h(n); optimal with an admissible heuristic
AO*	Heuristic search on AND-OR graphs; sum costs on AND arcs and revise costs upward
Minimax	MAX picks the highest and MIN the lowest value; assumes optimal play
Alpha-beta	Same answer as minimax, prune when $\alpha \geq \beta$; best case $O(b^{m/2})$
Evaluation function	Weighted sum of features used when the search is cut off

Frequently asked exam questions

- Explain BFS and DFS with algorithms, examples and a comparison of their properties.
- What is a heuristic? Explain hill climbing and its problems. How can they be overcome?
- Explain the A* algorithm with an example. When is A* optimal?
- Explain problem reduction and the AO* algorithm with an AND-OR graph.
- Explain the minimax algorithm. Write about optimal decisions in multiplayer games.
- Explain alpha-beta pruning with an example. What is an evaluation function?

ARTIFICIAL INTELLIGENCE • 23A31301T

UNIT III

Knowledge Representation and Reasoning

Digital Notes | Diagrams, worked examples and exam tips

SYLLABUS COVERED IN THIS UNIT

Knowledge representation issues, predicate logic, logic programming, semantic nets, frames and inheritance, constraint propagation, rule-based knowledge and deduction systems. Reasoning under uncertainty: probability review, Bayes' inference, Dempster-Shafer theory.

1. Knowledge Representation: Issues

An intelligent system must **know** things (facts, rules, relations) and be able to **use** that knowledge. **Knowledge representation (KR)** is the study of how to encode knowledge in a form a computer can store and reason with.

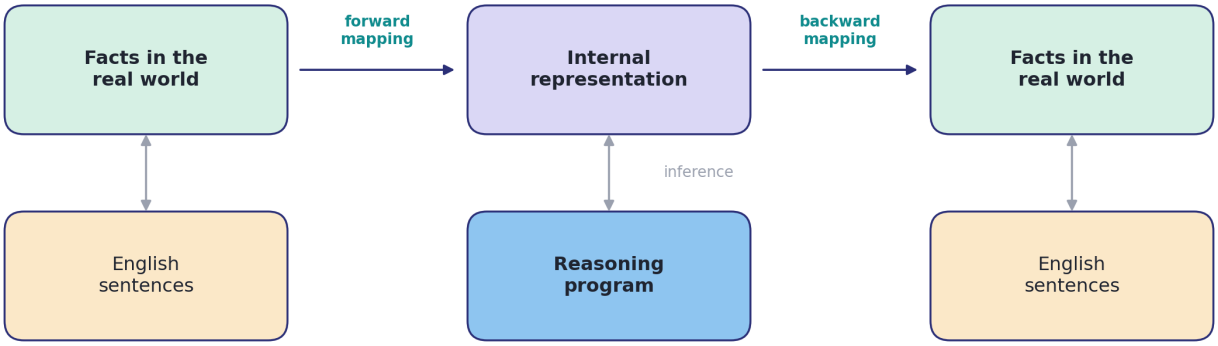


Fig 3.1 Mapping between facts and their representation: reasoning happens on the representation

What makes a good representation? Four properties

Property	Meaning
Representational adequacy	Can it represent *all* the kinds of knowledge needed in the domain?
Inferential adequacy	Can it derive *new* knowledge from the existing structures?
Inferential efficiency	Can the inference be guided toward the most promising directions?
Acquisitional efficiency	Can new knowledge be added easily (automatically if possible)?

Types of knowledge

- **Declarative:** facts about the world ("Paris is the capital of France").
- **Procedural:** how to do things (rules, plans, algorithms).
- **Meta-knowledge:** knowledge about knowledge (which rule to trust, how reliable a fact is).
- **Heuristic:** rules of thumb gained from experience.

Approaches to representing knowledge

Approach	Idea	Example
Simple relational	Facts stored as tables of attributes	A database of employees
Inheritable	Objects in a class hierarchy inherit properties	Semantic nets, frames
Inferential	Facts plus rules for deduction	Predicate logic, rules
Procedural	Knowledge embedded in executable code	Programs, production rules

Key issues in knowledge representation

- **Important attributes:** basic attributes such as *instance* and *isa* support inheritance and appear in almost every domain.
- **Relationships among attributes:** inverses (e.g., *parent / child*), existence in an isa hierarchy, single-valued attributes.
- **Granularity:** at what level of detail should knowledge be represented ("John spotted Sue" or a full description of vision)?
- **Representing sets of objects:** as one set (class) with properties, or individually?
- **Finding the right structure:** selecting the relevant piece of knowledge from a huge store when needed.

2. Predicate Logic

Predicate logic (first-order predicate calculus) is a precise, expressive language for knowledge. Where propositional logic only says "P is true", predicate logic describes **objects, their properties and relations**, and can state facts about *all* or *some* objects.

Quick recap: propositional logic

Propositional logic deals with whole statements (propositions) that are either true or false, combined with connectives. It is the foundation on which predicate logic is built.

P	Q	¬P	P ∧ Q	P ∨ Q	P → Q	P ↔ Q
T	T	F	T	T	T	T
T	F	F	F	T	F	F
F	T	T	F	T	T	F
F	F	T	F	F	T	T

Inference rule	Pattern	Example
Modus Ponens	P, P → Q ⊢ Q	It rains; if it rains the road is wet ⊢ road is wet
Modus Tollens	¬Q, P → Q ⊢ ¬P	Road is dry; if rain then wet ⊢ it is not raining
And-elimination	P ∧ Q ⊢ P	Ram is tall and strong ⊢ Ram is tall
Hypothetical syllogism	P → Q, Q → R ⊢ P → R	Study → pass; pass → degree ⊢ study → degree

Limitation: propositional logic cannot say "*all* students are smart" without listing every student. Predicate logic fixes this with variables and quantifiers.

Building blocks

Element	Notation	Example
Constants (objects)	Marcus, Caesar, 5	Marcus
Variables	x, y, z	x
Predicates (relations)	man(x), loves(x, y)	man(Marcus)
Functions	father(x)	father(Ravi)
Connectives	∧ and, ∨ or, ¬ not, → implies, ↔ iff	man(x) ∧ mortal(x)
Quantifiers	∀ for all, ∃ there exists	∀x man(x) → mortal(x)

REMEMBER
∀ usually goes with **→**: "All students are smart" = ∀x (student(x) → smart(x)).
∃ usually goes with **∧**: "Some student is smart" = ∃x (student(x) ∧ smart(x)).
Negation: ¬∀x P(x) ≡ ∃x ¬P(x) and ¬∃x P(x) ≡ ∀x ¬P(x).

English to predicate logic

English sentence	Predicate logic
Marcus was a man.	man(Marcus)
All Pompeians were Romans.	∀x: Pompeian(x) → Roman(x)
Every Roman was loyal to Caesar or hated him.	∀x: Roman(x) → loyalto(x, Caesar) ∨ hate(x, Caesar)
Everybody is loyal to someone.	∀x ∃y: loyalto(x, y)
Nobody likes taxes.	¬∃x: likes(x, taxes)

EXAMPLE
Inference by Modus Ponens. Given: (1) Pompeian(Marcus), (2) ∀x Pompeian(x) → Roman(x). Substitute x = Marcus in (2): Pompeian(Marcus) → Roman(Marcus). With (1), conclude **Roman(Marcus)**.

Logic programming (Prolog)

Logic programming uses a restricted form of logic (Horn clauses) as a programming language. The most famous language is **Prolog**. A program is a set of **facts** and **rules**; running it means asking a **query**, which the system answers by **backward chaining with backtracking** (search + unification).

```
parent(tom, bob).                 % fact: Tom is a parent of Bob
parent(bob, ann).                 % fact
grandparent(X, Z) :- parent(X, Y), parent(Y, Z).   % rule: X is grandparent of Z
                                                   %   IF X is parent of Y AND Y is parent of Z
?- grandparent(tom, W).           % query
W = ann.                          % answer
```

Prolog term	Meaning
Fact	A clause that is unconditionally true: *head.*
Rule	*head :- body.* Head is true if all goals in the body are true (**:-** means IF, comma means AND)
Query	*?- goal.* Asks whether the goal follows from the program, and finds variable bindings
Variable	Starts with a capital letter (X, Y); constants and predicates start with lower case

3. Semantic Nets

A **semantic network** is a graph of knowledge: **nodes** represent objects, concepts or situations, and **labelled arcs** represent the relationships between them. The two most important links are **is-a** (a class is a subclass of another) and **instance-of** (an object belongs to a class).

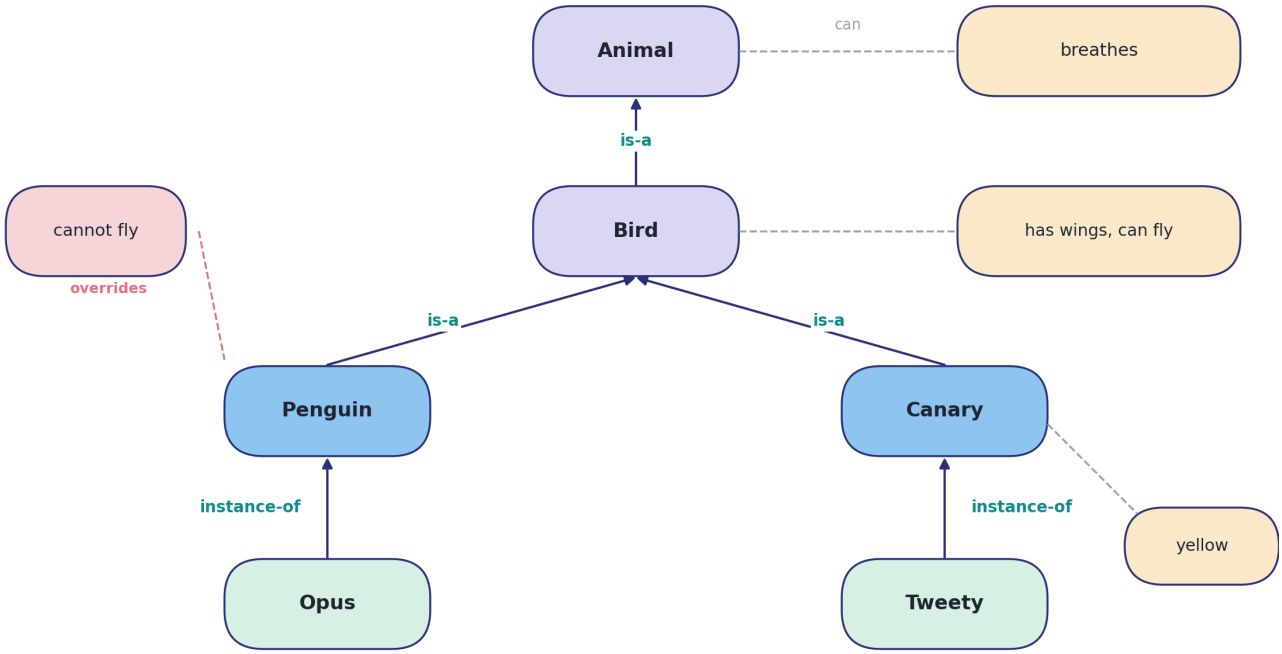


Fig 3.2 Semantic network: nodes are concepts/objects, labelled arcs are relations; properties are inherited down the is-a links

Inheritance

Properties stored at a general node are **inherited** by everything below it. Because Tweety is an instance of Canary, which is-a Bird, we can deduce *Tweety has wings and can fly* without storing it separately. A more specific node can **override** an inherited value: Penguin is-a Bird but *cannot fly*, so Opus does not fly.

Advantages	Limitations
Easy to visualise; natural for hierarchies	No standard meaning for links, so interpretation can be ambiguous
Efficient inheritance reduces storage	Hard to represent negation, disjunction and quantifiers properly
Quick access to related concepts by following links	Exceptions and defaults can cause inconsistent conclusions

4. Frames and Inheritance

A **frame** (Marvin Minsky, 1975) is a structured record describing a stereotyped object or situation. It is like a class: it has a **name** and a collection of **slots** (attributes), and each slot holds a value and possibly extra information called **facets**.

Facet	Purpose
Value	The actual filler of the slot
Default	Value assumed when no other value is known
Range / type	Restricts which values are legal
If-needed (procedure)	Code executed to *compute* the value when asked
If-added / if-removed (demon)	Code triggered automatically when the value changes

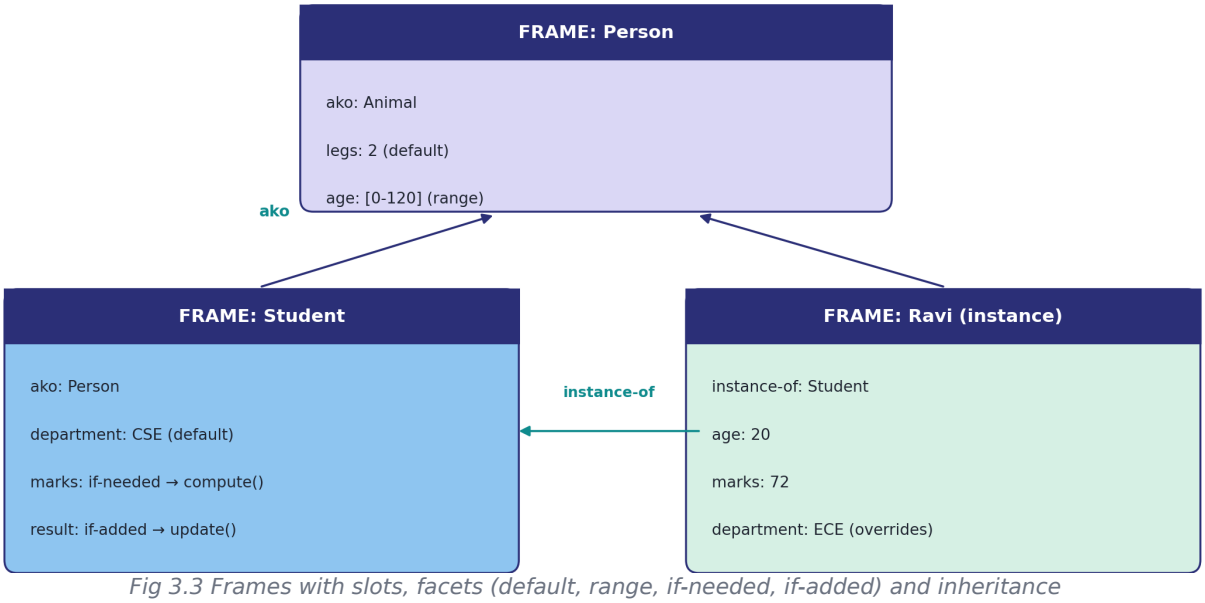


Fig 3.3 Frames with slots, facets (default, range, if-needed, if-added) and inheritance

- **Inheritance:** a child frame (linked by *ako*, 'a kind of', or *instance-of*) inherits slots and defaults from its parent frames. Local values override inherited defaults.
- **Example:** Ravi is an instance of Student, so he inherits *legs = 2* from Person. His department is stored locally as ECE, which overrides the default CSE.
- **Frames vs semantic nets:** frames group all information about one object in one place and add procedures and defaults; semantic nets spread information over graph links.

Choosing a representation: quick comparison

Scheme	Best for	Weakness
Predicate logic	Precise facts and sound deduction	Can be slow; hard to handle exceptions/uncertainty
Semantic nets	Class hierarchies, relations between concepts	No formal semantics; poor at quantifiers
Frames	Stereotyped objects and situations with defaults	Rigid structure; defaults can conflict
Production rules	Expert knowledge, modular and easy to edit	Large rule bases are hard to maintain
Probabilistic models	Reasoning with uncertain evidence	Needs many probabilities

5. Constraint Propagation

Many problems (timetabling, map colouring, Sudoku, scheduling) are **constraint satisfaction problems (CSPs)**. A CSP has three parts:

- **Variables** $X_1 \ldots X_n$
- **Domains** D_i, the set of possible values of each variable
- **Constraints**, rules saying which combinations of values are allowed

Constraint propagation uses the constraints to **reduce the domains** of variables: whenever a value is assigned or removed, its consequences are spread to neighbouring variables, pruning values that can no longer be part of a solution. This often finds the answer with little or no search.

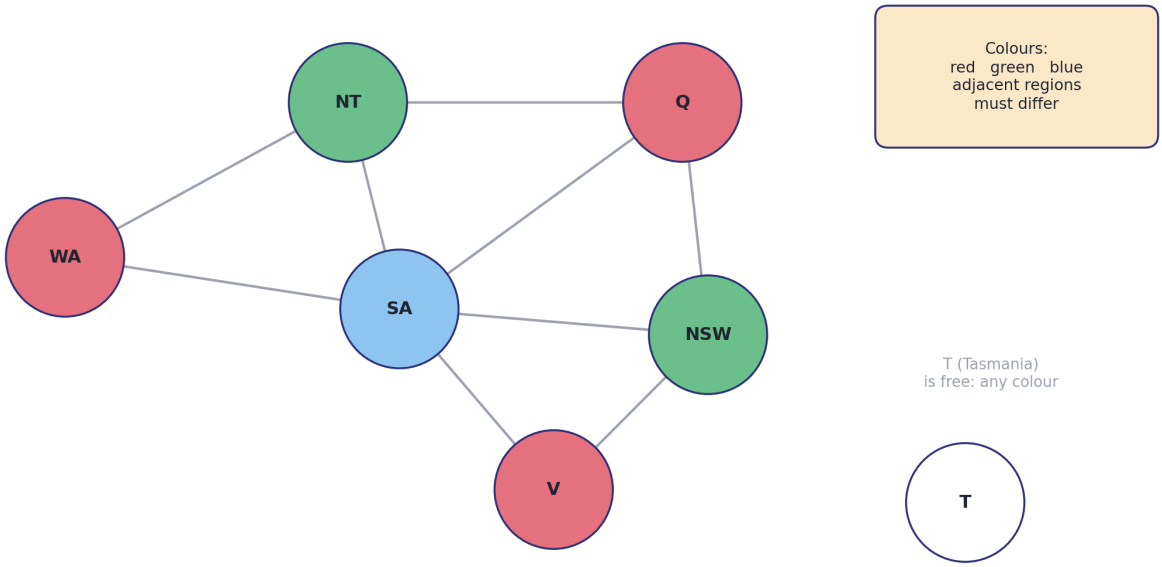


Fig 3.4 Constraint graph for map colouring (nodes = variables, edges = 'must differ' constraints), shown with a valid solution

Step	Action	Domains after propagation
0	Start	All variables {R, G, B}
1	Assign WA = R	NT {G, B}; SA {G, B}
2	Assign NT = G	SA {B}; Q {R, B}
3	SA is forced to B	Q {R}; NSW {R, G}; V {R, G}
4	Q is forced to R	NSW {G}
5	NSW is forced to G	V {R} (T may be any colour)

- **Forward checking:** after assigning a variable, delete inconsistent values from its neighbours' domains. If any domain becomes empty, **backtrack** immediately.
- **Arc consistency:** an arc X → Y is consistent if for *every* value of X there is some allowed value of Y. Inconsistent values are deleted repeatedly (the AC-3 algorithm) until all arcs are consistent.
- **Heuristics:** choose the variable with the *fewest remaining values* (MRV) first, and try the least constraining value first.

EXAM TIP
Constraint propagation = **inference** that narrows domains. Backtracking = **search**. Real CSP solvers interleave both.

6. Representing Knowledge Using Rules

The most popular way to hold expert knowledge is the **production rule**, an IF-THEN statement: **IF** conditions (antecedent) **THEN** action or conclusion (consequent). A system built from such rules is a **production system**.

EXAMPLE
R1: IF has_feathers AND lays_eggs THEN bird
R2: IF bird AND cannot_fly AND swims THEN penguin
R3: IF fever AND cough THEN suspect_flu

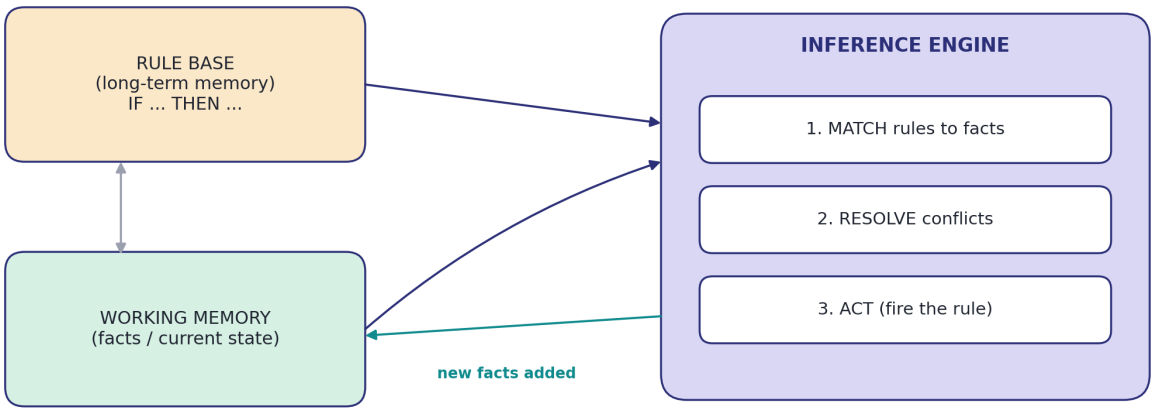


Fig 3.5 Architecture of a production (rule-based) system: match, resolve, act

Components of a production system

- **Rule base** (production memory): the set of IF-THEN rules, i.e., the permanent knowledge.
- **Working memory:** the facts that are currently known; it changes as rules fire.
- **Inference engine** (control strategy): repeatedly **matches** rules against the working memory, picks one to fire (**conflict resolution**), and **acts** on it.

Conflict resolution strategies

Strategy	Idea
Specificity	Prefer the rule with more conditions (the more specific rule)
Recency	Prefer rules that use the most recently added facts
Rule order	Choose the first matching rule in the list (priority)
Refractoriness	Never fire the same rule twice on the same facts

7. Rule-Based Deduction Systems

These systems derive conclusions from the rules using one of two directions of reasoning.

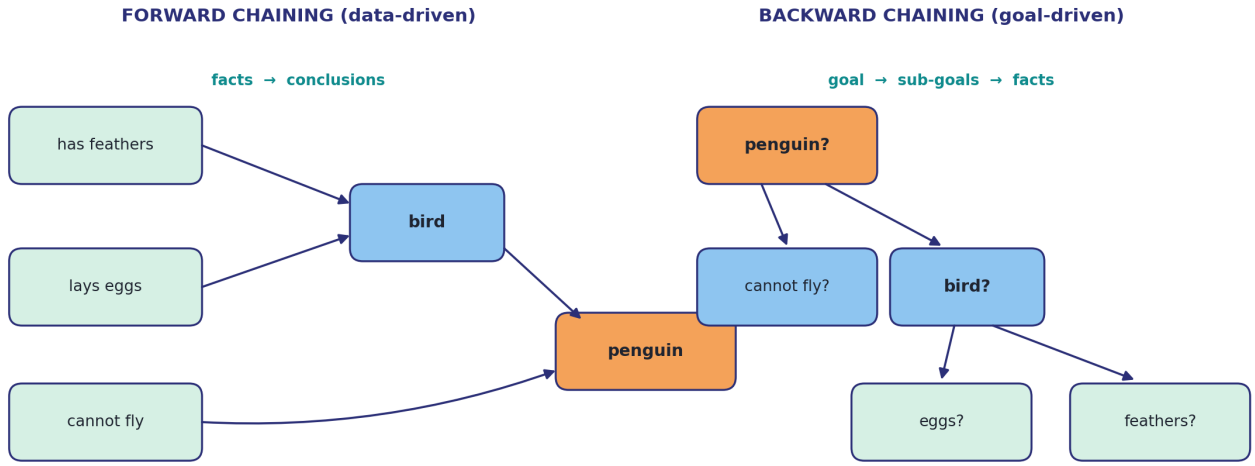


Fig 3.6 Forward vs backward chaining on the same rules

	Forward chaining	Backward chaining
Direction	From known facts to conclusions	From a goal back to supporting facts
Also called	Data-driven, bottom-up	Goal-driven, top-down
Starts with	Facts in working memory	A hypothesis (goal) to prove
Good when	Many facts, no single goal: monitoring, design, planning	A specific goal: diagnosis, 'is X true?'
Example systems	XCON (R1), OPS5	MYCIN, Prolog

EXAMPLE
Forward: facts {has_feathers, lays_eggs, cannot_fly, swims}. R1 fires → add *bird*. R2 fires → add *penguin*.
Backward: goal *penguin?* → needs *bird*, *cannot_fly*, *swims*. To prove *bird* → R1 needs has_feathers and lays_eggs, both in facts. Goal proved.

8. Reasoning under Uncertainty

Real-world knowledge is rarely certain. A doctor sees symptoms without being sure of the disease; a sensor gives noisy readings. Pure logic (true/false) cannot cope, so AI uses methods that handle **degrees of belief**.

Sources of uncertainty	Example
Incomplete information	Not all test results are available
Noisy / unreliable data	A faulty sensor reading
Imprecise knowledge	"Fever usually means infection"
Randomness in the world	Will it rain tomorrow?

8.1 Review of probability

- **Probability P(A)** $\in [0, 1]$. P(true) = 1, P(false) = 0, P(¬A) = 1 − P(A). All outcomes sum to 1.
- **Joint probability** P(A ∧ B): both occur. **Union:** P(A ∨ B) = P(A) + P(B) − P(A ∧ B).
- **Conditional probability:** P(A | B) = P(A ∧ B) / P(B), the chance of A once B is known.
- **Independence:** A and B are independent if P(A ∧ B) = P(A)·P(B), equivalently P(A | B) = P(A).
- **Product rule:** P(A ∧ B) = P(A | B)·P(B).

8.2 Bayes' theorem and probabilistic inference

FORMULA

P(H | E) = P(E | H) · P(H) / P(E)

P(H) = **prior** belief in hypothesis H | P(E | H) = **likelihood** of the evidence if H is true | P(H | E) = **posterior** belief after seeing E.

P(E) = P(E | H)P(H) + P(E | ¬H)P(¬H)

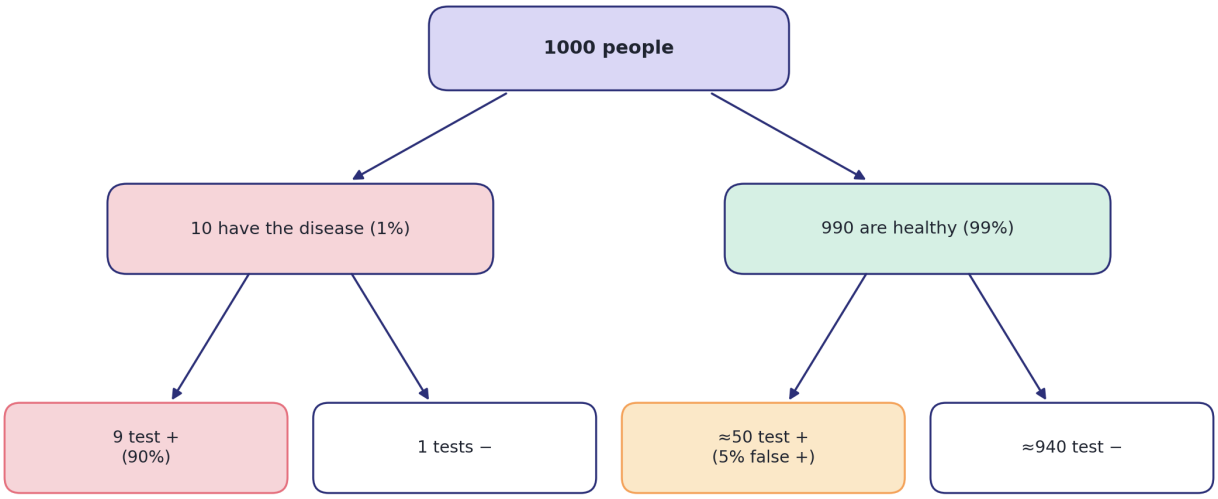
Worked example. 1% of people have a disease. A test is positive for 90% of sick people (sensitivity) and for 5% of healthy people (false positives). A person tests positive. What is the chance they are sick?

SOLUTION

P(D) = 0.01, P(+ | D) = 0.90, P(+ | ¬D) = 0.05.

P(+) = 0.90 × 0.01 + 0.05 × 0.99 = 0.009 + 0.0495 = 0.0585.

P(D | +) = 0.009 / 0.0585 ≈ 0.154 (about 15%). Despite a 90% accurate test, most positives are false alarms because the disease is rare (low prior).



Positives = 9 + 50 = 59. Truly diseased among them = 9/59 ≈ 15% (not 90%!)

Fig 3.7 Natural-frequency view of Bayes' theorem (disease 1%, sensitivity 90%, false-positive rate 5%)

EXAMPLE

Factory problem. Machine M1 makes 60% of items with 3% defective; M2 makes 40% with 5% defective. An item is found defective. Which machine made it?
P(D) = 0.6×0.03 + 0.4×0.05 = 0.018 + 0.020 = 0.038. P(M1 | D) = 0.018/0.038 ≈ **0.474**; P(M2 | D) = 0.020/0.038 ≈ **0.526**.

Naive Bayes: with several pieces of evidence $E_1 \ldots E_n$ assumed independent given H, $P(H \mid E_1 \ldots E_n) \propto P(H) \cdot \Pi P(E_i \mid H)$. It is simple and works surprisingly well (spam filters, diagnosis).

Bayesian (belief) networks

A **Bayesian network** is a directed acyclic graph in which each node is a random variable and each arrow means 'directly influences'. Each node stores a **conditional probability table** given its parents. The network gives the full joint distribution compactly: $\mathbf{P(x_1 \ldots x_n) = \Pi\ P(x_i \mid parents(x_i))}$.

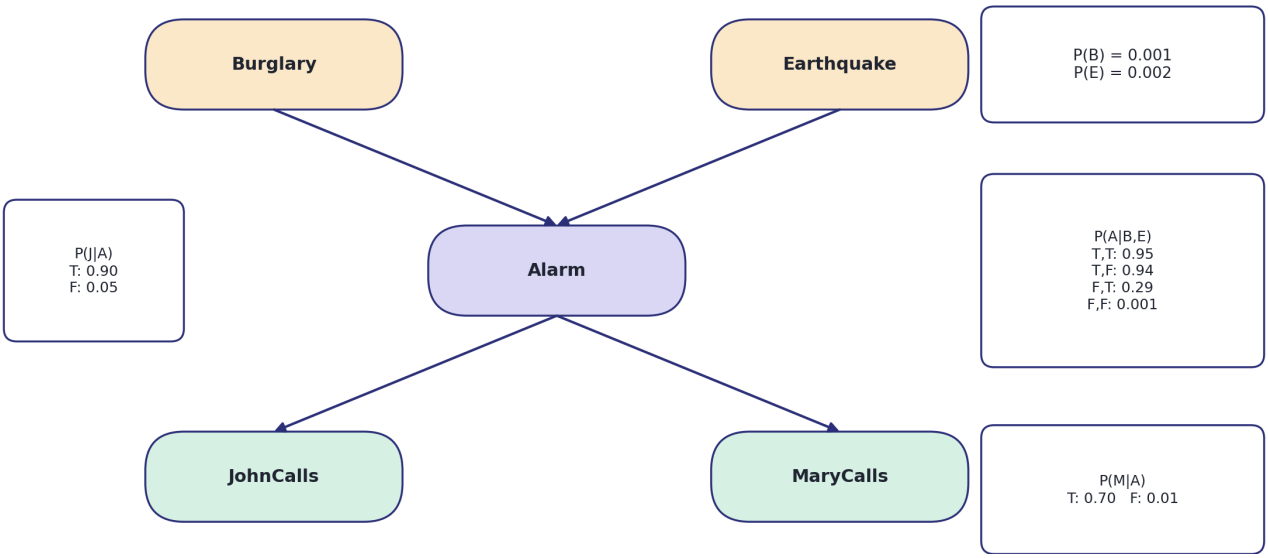


Fig 3.8 A Bayesian network: arrows show direct influence; each node stores a conditional probability table (CPT)

EXAMPLE

P(John calls, Mary calls, alarm, no burglary, no earthquake) = P(J|A)·P(M|A)·P(A|¬B,¬E)·P(¬B)·P(¬E) = 0.90 × 0.70 × 0.001 × 0.999 × 0.998 ≈ **0.00063**.

8.3 Dempster-Shafer theory

Bayes' theory forces you to assign probabilities to *every* hypothesis, even when you know nothing. **Dempster-Shafer (D-S) theory** allows you to say 'I don't know' by assigning belief to **sets** of hypotheses and keeping the leftover as **ignorance**.

Term	Meaning
Frame of discernment Θ	The set of all mutually exclusive possible answers, e.g., Θ = {A, B}
Mass function m	Assigns a mass to each subset of Θ; masses sum to 1 and m(∅) = 0. m(Θ) represents ignorance
Belief Bel(X)	Total mass of all subsets of X: the evidence that *supports* X (lower bound)
Plausibility Pl(X)	1 − Bel(¬X): the evidence that does *not rule out* X (upper bound)
Belief interval	[Bel(X), Pl(X)]: the width Pl − Bel measures the uncertainty about X

Dempster's rule of combination merges two independent bodies of evidence: $m(Z) = \Sigma_{X \cap Y = Z} m_1(X) \cdot m_2(Y) / (1 - K)$, where K = the total mass that falls on empty intersections (the **conflict**).

EXAMPLE

Θ = {A, B}. Evidence 1: $m_1(\{A\}) = 0.6$, $m_1(\Theta) = 0.4$. Evidence 2: $m_2(\{A\}) = 0.5$, $m_2(\{B\}) = 0.3$, $m_2(\Theta) = 0.2$.

Conflict: {A}∩{B} = ∅, so K = 0.6 × 0.3 = 0.18, and 1 − K = 0.82.

m({A}) = (0.6×0.5 + 0.6×0.2 + 0.4×0.5) / 0.82 = 0.62/0.82 ≈ **0.756**; m({B}) = (0.4×0.3)/0.82 ≈ **0.146**; m(Θ) = (0.4×0.2)/0.82 ≈ **0.098**.

Bel(A) = 0.756; Pl(A) = 1 − Bel(B) = 1 − 0.146 = 0.854. Belief interval for A = **[0.756, 0.854]**.

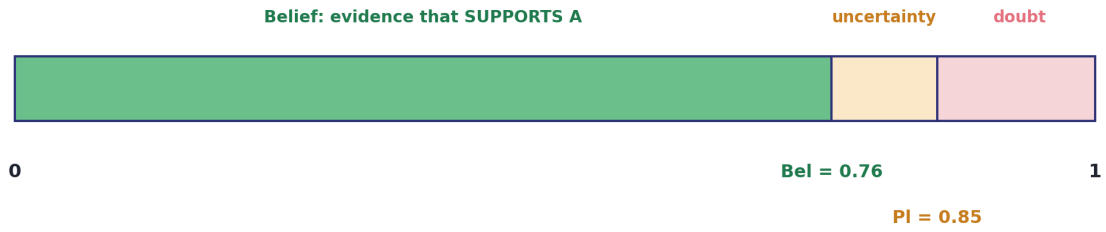


Fig 3.9 Belief interval [Bel(A), Pl(A)] for the Dempster-Shafer example

	Bayesian probability	Dempster-Shafer
Ignorance	Must be spread over hypotheses (e.g., 50/50)	Kept explicitly as mass on Θ
Output	A single probability P(H)	An interval [Bel, Pl]
Evidence about sets	Only about single outcomes	Can be assigned to sets of outcomes
Needs	Prior probabilities and likelihoods	Mass functions only

Quick Revision: Unit III at a Glance

Topic	One-line recall
KR properties	Representational and inferential adequacy, inferential efficiency, acquisitional efficiency
Predicate logic	Objects, predicates, ∀ with →, ∃ with ∧; reasoning by Modus Ponens
Prolog	Facts + rules (head :- body) + queries, solved by backward chaining
Semantic net / frame	Graph of concepts with is-a links / structured slots with defaults and demons; both inherit
Constraint propagation	Spread constraints to shrink domains; forward checking, arc consistency
Production system	Rule base + working memory + inference engine (match, resolve, act)
Forward / backward	Data-driven from facts / goal-driven from hypothesis
Bayes	P(H|E) = P(E|H)P(H)/P(E)
Dempster-Shafer	Masses on sets; belief interval [Bel, Pl]; combine with Dempster's rule

Frequently asked exam questions

- Discuss the issues in knowledge representation and the properties of a good KR scheme.
- Represent given English statements in predicate logic. Write a short Prolog program.
- Explain semantic nets and frames with examples. How does inheritance work?
- What is constraint propagation? Explain with a map-colouring example.
- Explain a rule-based deduction system. Compare forward and backward chaining.
- State Bayes' theorem and solve a numerical problem. Explain Dempster-Shafer theory with an example.

ARTIFICIAL INTELLIGENCE • 23A31301T

UNIT IV

First-Order Logic, Inference and Learning

Digital Notes | Diagrams, worked examples and exam tips

SYLLABUS COVERED IN THIS UNIT

First-order logic, inference in FOL, propositional vs first-order inference, unification and lifting, forward chaining, backward chaining, resolution. Learning from observation: inductive learning, decision trees, explanation-based learning, statistical learning methods, reinforcement learning.

1. First-Order Logic (FOL)

First-order logic (also called predicate logic) represents the world as **objects**, **properties** of objects and **relations** between them. It is far more expressive than propositional logic, which has only true/false facts.

Symbol type	Examples	Meaning
Constants	John, Richard, Crown	Names of specific objects
Variables	x, y, z	Stand for any object
Predicates	King(x), Brother(x, y)	Properties and relations (true/false)
Functions	LeftLeg(x), Mother(x)	Map objects to objects
Connectives	¬ ∧ ∨ → ↔	Combine sentences
Quantifiers	∀ (for all), ∃ (there exists)	Make statements about groups of objects
Equality	Mother(John) = Mary	Two terms refer to the same object

Terms, atoms and sentences

- **Term:** an expression that refers to an object: a constant, a variable or a function applied to terms, e.g., *Mother(John)*.
- **Atomic sentence:** a predicate applied to terms, e.g., *Brother(Richard, John)*.
- **Complex sentence:** atomic sentences joined by connectives or quantified, e.g., ∀*x King(x)* ∧ *Greedy(x)* → *Evil(x)*.
- **Ground term / sentence:** contains no variables. A sentence with no free variables is **closed**.

English	FOL
All kings are persons.	∀x King(x) → Person(x)
Some student is clever.	∃x Student(x) ∧ Clever(x)
Every student takes at least one course.	∀x Student(x) → ∃y Course(y) ∧ Takes(x, y)
Brothers are siblings.	∀x, y Brother(x, y) → Sibling(x, y)
No one likes everybody.	¬∃x ∀y Likes(x, y)

REMEMBER
∀ pairs with → and **∃ pairs with ∧**. Writing ∀x King(x) ∧ Person(x) wrongly says *everything* is a king, and ∃x King(x) → Person(x) is trivially true if anything is not a king. The order of mixed quantifiers matters: ∀x ∃y Loves(x, y) (everyone loves someone) is not the same as ∃y ∀x Loves(x, y) (someone is loved by everyone).

2. Inference in First-Order Logic

2.1 Reducing FOL to propositional inference

To use fast propositional methods on FOL knowledge, we first **instantiate** the quantifiers.

Rule	Statement	Example
Universal Instantiation (UI)	From ∀v α, infer α with v replaced by any ground term g	∀x King(x) ∧ Greedy(x) → Evil(x) gives King(John) ∧ Greedy(John) → Evil(John)
Existential Instantiation (EI)	From ∃v α, infer α with v replaced by a **new** constant (Skolem constant) K	∃x Crown(x) ∧ OnHead(x, John) gives Crown(C1) ∧ OnHead(C1, John), where C1 is new

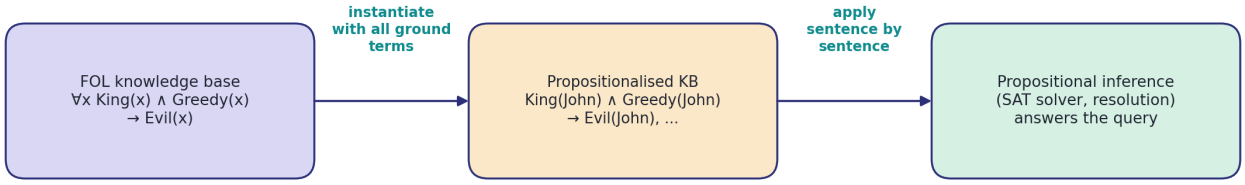


Fig 4.1 Inference by propositionalisation: convert FOL to propositional logic, then reason

Propositionalisation: replace every quantified sentence with all its ground instances, then apply a propositional inference algorithm. It works (Herbrand's theorem: if a sentence is entailed, a finite proof exists), but with function symbols there are infinitely many ground terms, and the method **generates huge numbers of irrelevant sentences**.

	Propositional inference	First-order inference
Expressiveness	Facts only: P, Q, R	Objects, relations, quantifiers
Variables	None	Yes; handled by unification
Knowledge size	Needs a separate sentence for each object	One rule covers all objects
Decidability	Decidable (finite truth tables)	Semi-decidable: if a conclusion is entailed it can be proved, but non-entailment may run forever
Typical method	Truth tables, DPLL / SAT, resolution	Unification + Generalised Modus Ponens, resolution

2.2 Unification and lifting

Instead of generating all instances, **lifted inference** works directly on sentences with variables. It uses **unification** to find substitutions that make different expressions identical.

DEFINITION
Substitution θ = a set of variable bindings such as {x/John, y/Mary}. **Unifier** of two sentences p and q: a substitution θ such that SUBST(θ, p) = SUBST(θ, q). The **most general unifier (MGU)** is the one that places the fewest restrictions on the variables.

Generalised Modus Ponens (GMP) is the lifted version of Modus Ponens. For atomic sentences p_i, p_i', q and a substitution θ with SUBST(θ, p_i') = SUBST(θ, p_i) for all i:

RULE
$$p_1', p_2', \ldots, p_n', (p_1 \wedge p_2 \wedge \ldots \wedge p_n \Rightarrow q) \vdash \text{SUBST}(\theta, q)$$

Sentence 1	Sentence 2	Unifier θ
Knows(John, x)	Knows(John, Jane)	{x / Jane}
Knows(John, x)	Knows(y, Bill)	{x / Bill, y / John}
Knows(John, x)	Knows(y, Mother(y))	{y / John, x / Mother(John)}
Knows(John, x)	Knows(x, Elizabeth)	**Fails**: x cannot be John and Elizabeth at once. Fix: *standardise apart* to Knows(x_{17}, Elizabeth), giving {x / Elizabeth, x_{17} / John}
P(x, f(x))	P(a, f(a))	{x / a}

Unification algorithm (idea)

- **1.** If the two expressions are identical, return the empty substitution. If either is a variable, bind it to the other (after the **occur check**).
- **2.** If both are compound (same predicate/function name and arity), unify their arguments **left to right**, applying each new binding to the remaining arguments.
- **3.** Otherwise (different predicates, different constants, arity mismatch) return **failure**.
- **Occur check:** a variable x may not be bound to a term that contains x (e.g., x with f(x) fails), otherwise we would build an infinite term.

3. Forward Chaining

Forward chaining starts from the known **facts** and applies Generalised Modus Ponens to every rule whose premises are satisfied, adding the conclusions as new facts. It stops when the query is derived or no new fact can be generated. It is **data-driven**.

```
function FOL-FC-ASK(KB, query) returns a substitution or false
   repeat until new is empty
       new <- { }
       for each rule (p1 ^ ... ^ pn => q) in KB (standardised apart) do
           for each theta such that SUBST(theta, p1 ^ ... ^ pn) = SUBST(theta, p1' ^ ... ^ pn')
                                    for some facts p1', ..., pn' in KB
               q' <- SUBST(theta, q)
               if q' does not unify with an existing sentence in KB or new then
                   add q' to new;   phi <- UNIFY(q', query)
                   if phi is not fail then return phi
       add new to KB
   return false
```

EXAMPLE

The law says it is a crime for an American to sell weapons to hostile nations. Nono, an enemy of America, has some missiles, and all of them were sold to it by Colonel West, an American. Prove: West is a criminal.

R1: American(x) ∧ Weapon(y) ∧ Sells(x, y, z) ∧ Hostile(z) → Criminal(x)
R2: Missile(x) ∧ Owns(Nono, x) → Sells(West, x, Nono) R3: Missile(x) → Weapon(x) R4: Enemy(x, America) → Hostile(x)
Facts: American(West), Enemy(Nono, America), Owns(Nono, M1), Missile(M1)

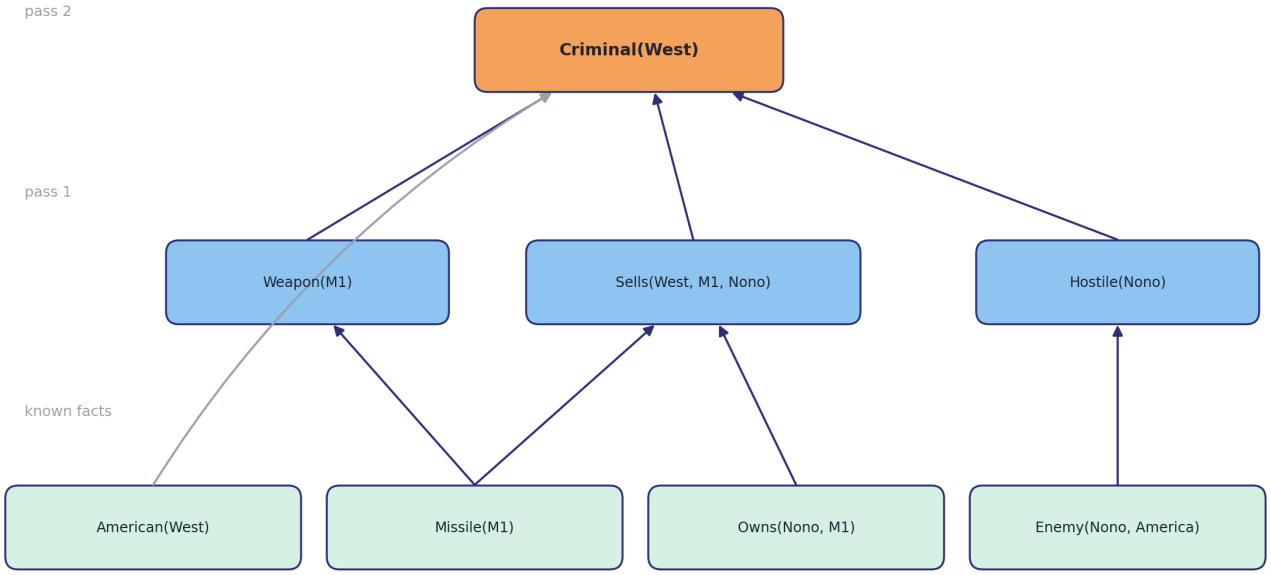


Fig 4.2 Forward chaining proof for 'West is a criminal': new facts are added pass by pass until the query appears

Property	Forward chaining
Sound	Yes: every step is a valid inference
Complete	Yes, for **definite clauses** (at most one positive literal) in a function-free KB (Datalog); terminates in a finite number of steps
Weakness	May derive many irrelevant facts that have nothing to do with the query
Used in	Production systems (OPS5, XCON), deductive databases, monitoring and planning

4. Backward Chaining

Backward chaining starts from the **goal**. It finds rules whose conclusion unifies with the goal and makes their premises the new **sub-goals**, proving them recursively (depth-first, left to right) until all sub-goals match known facts. It is **goal-driven**, the strategy used by Prolog and MYCIN.

```
function FOL-BC-ASK(KB, goals, theta) returns a set of substitutions
   if goals is empty then return { theta }
   q' <- SUBST(theta, FIRST(goals));   answers <- { }
   for each rule (p1 ^ ... ^ pn => q) in KB whose head q unifies with q' using theta':
        new_goals <- [p1, ..., pn | REST(goals)]
        answers <- FOL-BC-ASK(KB, new_goals, COMPOSE(theta', theta)) U answers
   return answers
```

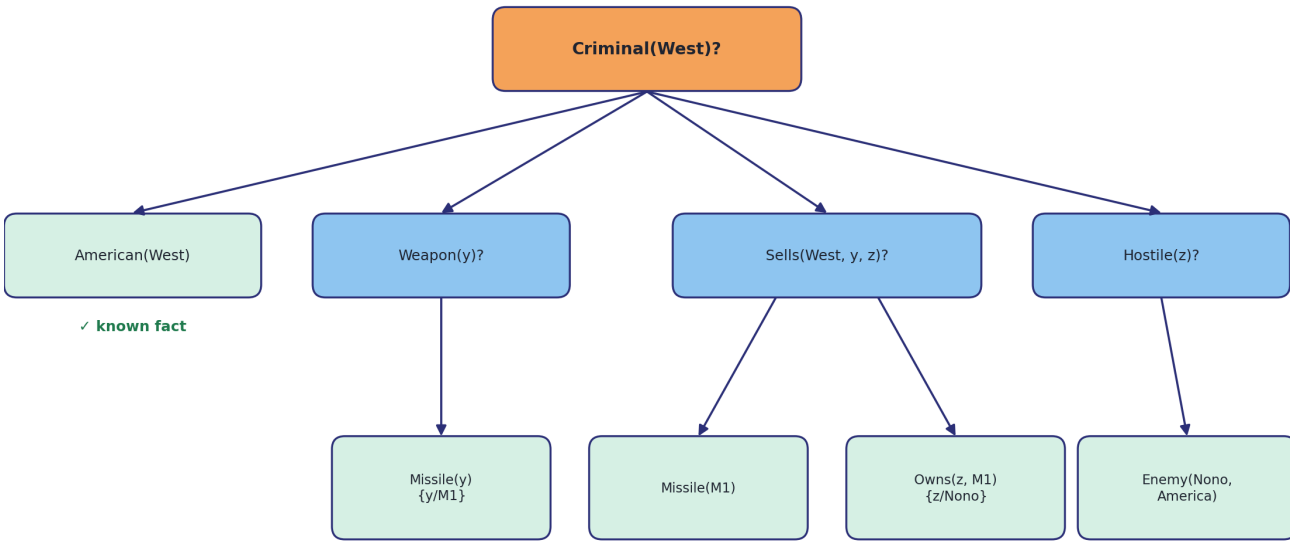


Fig 4.3 Backward chaining: start from the goal, create sub-goals, bind variables, end at known facts

	Forward chaining	Backward chaining
Starts from	Known facts	Query / goal
Efficiency	May produce irrelevant facts	Focused on the goal, so fewer irrelevant steps
Search style	Breadth-style, bottom-up	Depth-first with backtracking, top-down
Risk	Combinatorial explosion of facts	Infinite loops on recursive rules; repeated sub-goals
Typical use	Monitoring, control, configuration	Diagnosis, question answering, Prolog

5. Resolution

Resolution is a single, powerful inference rule that, used with **refutation**, gives a **complete** inference procedure for first-order logic: if the knowledge base entails a sentence, resolution can prove it. It works on sentences in **conjunctive normal form (CNF)**: a conjunction of clauses, where each clause is a disjunction of literals.

RULE
Resolution rule: from $(l_1 \vee \ldots \vee l_k)$ and $(m_1 \vee \ldots \vee m_n)$ where l_i unifies with $\neg m_j$ (unifier θ), infer
SUBST(θ, $l_1 \vee \ldots \vee l_{i-1} \vee l_{i+1} \vee \ldots \vee l_k \vee m_1 \vee \ldots \vee m_{j-1} \vee m_{j+1} \vee \ldots \vee m_n$)
Example: (¬Student(x) ∨ Studies(x)) and Student(Ravi) resolve with {x/Ravi} to give Studies(Ravi).

Converting a sentence to CNF: the six steps

Step	What to do	Example
1	Eliminate ↔ and →: P → Q becomes ¬P ∨ Q	∀x Man(x) → Mortal(x) ⇒ ∀x ¬Man(x) ∨ Mortal(x)
2	Move ¬ inwards (De Morgan, quantifier rules)	¬∀x P ⇒ ∃x ¬P; ¬(P ∧ Q) ⇒ ¬P ∨ ¬Q
3	Standardise variables: give each quantifier its own variable	∀x P(x) ∨ ∃x Q(x) ⇒ ∀x P(x) ∨ ∃y Q(y)
4	**Skolemise**: replace each ∃ variable with a Skolem constant/function of the enclosing ∀ variables	∀x ∃y Loves(x, y) ⇒ ∀x Loves(x, F(x))
5	Drop the universal quantifiers	∀x ¬Man(x) ∨ Mortal(x) ⇒ ¬Man(x) ∨ Mortal(x)
6	Distribute ∨ over ∧ to get clauses	(A ∧ B) ∨ C ⇒ (A ∨ C) ∧ (B ∨ C)

Proof by refutation (contradiction)

To prove a goal α from a knowledge base KB: **(1)** convert KB into clauses; **(2)** add the clauses of **¬α**; **(3)** repeatedly resolve pairs of clauses and add the results; **(4)** if the **empty clause (□)** is produced, KB ∧ ¬α is contradictory, so α is proved.

EXAMPLE

KB: (1) ¬Student(x) ∨ Studies(x) (2) ¬Studies(y) ∨ Passes(y) (3) Student(Ravi). **Goal:** Passes(Ravi). Negate the goal: (4) ¬Passes(Ravi). Resolve (4) with (2), {y/Ravi}: (5) ¬Studies(Ravi). Resolve (5) with (1), {x/Ravi}: (6) ¬Student(Ravi). Resolve (6) with (3): □. Proved.

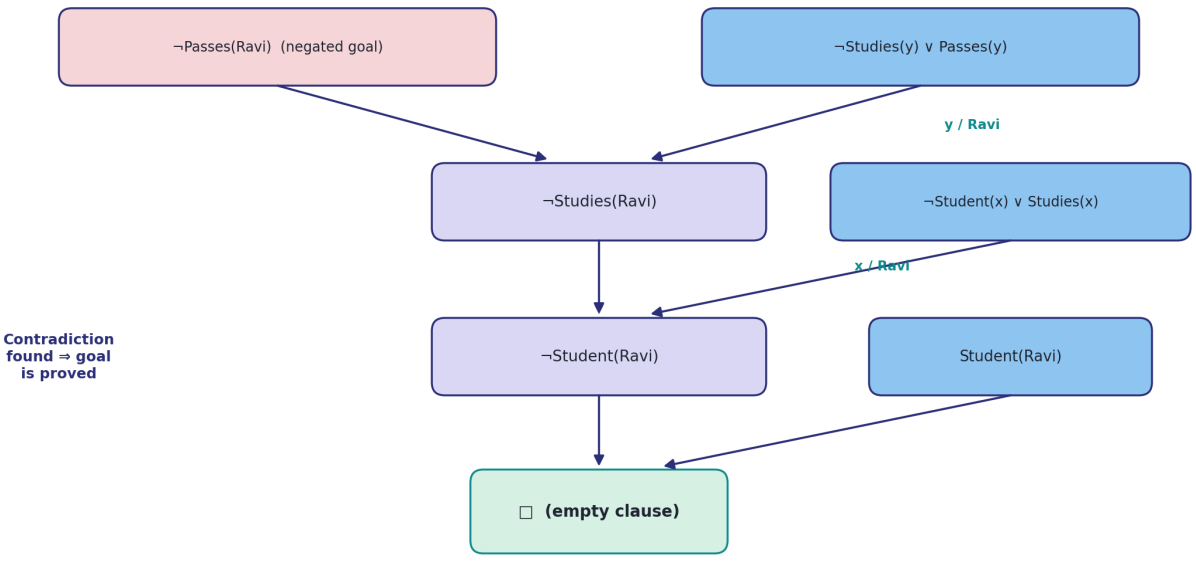


Fig 4.4 Resolution refutation: the negated goal leads to the empty clause, so Passes(Ravi) follows from the KB

- **Sound** (never derives something false) and **refutation-complete** (Gödel/Robinson).
- **Strategies to reduce search:** *unit preference* (resolve with single-literal clauses first), *set of support* (always use a clause from the negated goal or its descendants), *input resolution*, subsumption (delete redundant clauses).

6. Learning from Observation

A **learning agent** improves its performance on future tasks after making observations about the world. Learning is needed when the designer cannot anticipate every situation or when the environment changes.

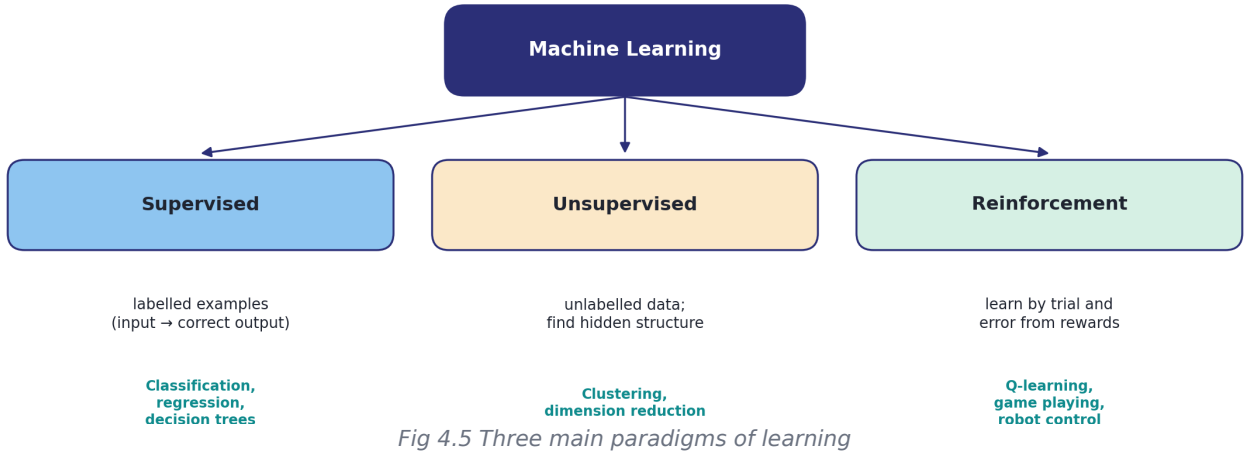


Fig 4.5 Three main paradigms of learning

6.1 Inductive learning

Inductive learning (supervised) learns a function **f** from **examples** (x, f(x)). The goal is a **hypothesis h** that approximates f and **generalises**, i.e., predicts well on new, unseen inputs.

- **Hypothesis space H:** the set of functions the learner can choose from. A hypothesis is **consistent** if it agrees with all training examples.
- **Bias:** any preference for one hypothesis over another. **Ockham's razor:** prefer the simplest hypothesis that is consistent with the data.
- **Overfitting:** a complex h fits noise in the training data and generalises badly. **Underfitting:** h is too simple to capture the pattern.
- **Evaluation:** hold out a **test set**, or use cross-validation; the learning curve shows accuracy as the training set grows.

7. Decision Trees

A **decision tree** takes a vector of attribute values and returns a decision. **Internal nodes** test an attribute, **branches** carry the test outcomes and **leaves** give the class. Trees are easy to read and can be written as IF-THEN rules.

ID3 / decision-tree learning builds the tree top-down. At each step, pick the attribute that best splits the examples, using **information gain** based on **entropy**.

FORMULA

Entropy(S) = $-\,p_+ \log_2 p_+ - p_- \log_2 p_-$ (0 = pure set, 1 = 50/50 mix)

Gain(S, A) = Entropy(S) $-\,\Sigma_{v \in \text{values}(A)}$ ($|S_v| / |S|$) · Entropy(S_v)

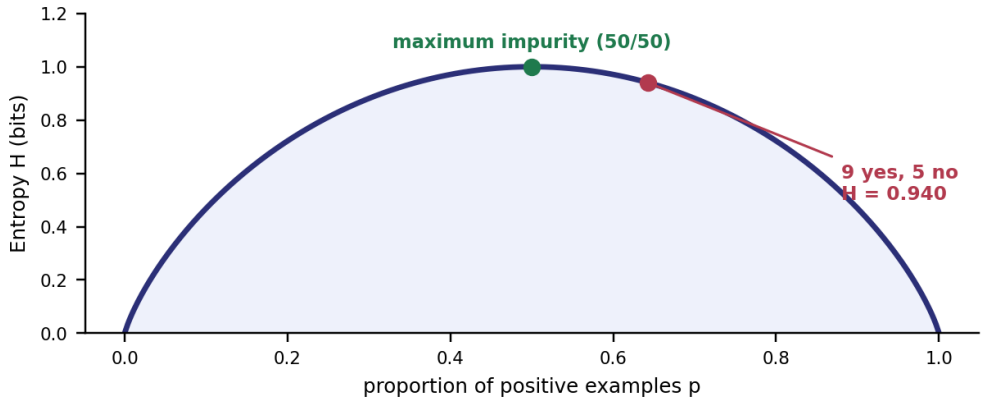


Fig 4.7 Binary entropy: highest when classes are evenly mixed, zero when pure

Worked example: Play Tennis. Decide whether to play tennis (14 days, 9 Yes and 5 No).

Day	Outlook	Temp	Humidity	Wind	Play
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

SOLUTION

Step 1. Entropy(S) = $-(9/14)\log_2(9/14) - (5/14)\log_2(5/14)$ = **0.940**.

Step 2. Gain(Outlook): Sunny (2 Yes, 3 No) = 0.971; Overcast (4 Yes, 0 No) = 0; Rain (3 Yes, 2 No) = 0.971. Gain = 0.940 − (5/14)(0.971) − (4/14)(0) − (5/14)(0.971) = **0.247**.

Similarly Gain(Humidity) = 0.151, Gain(Wind) = 0.048, Gain(Temperature) = 0.029.

Step 3. Outlook has the highest gain, so it becomes the **root**. Overcast is all Yes (a leaf). Repeat for the Sunny and Rain subsets: Sunny splits best on Humidity, Rain on Wind.

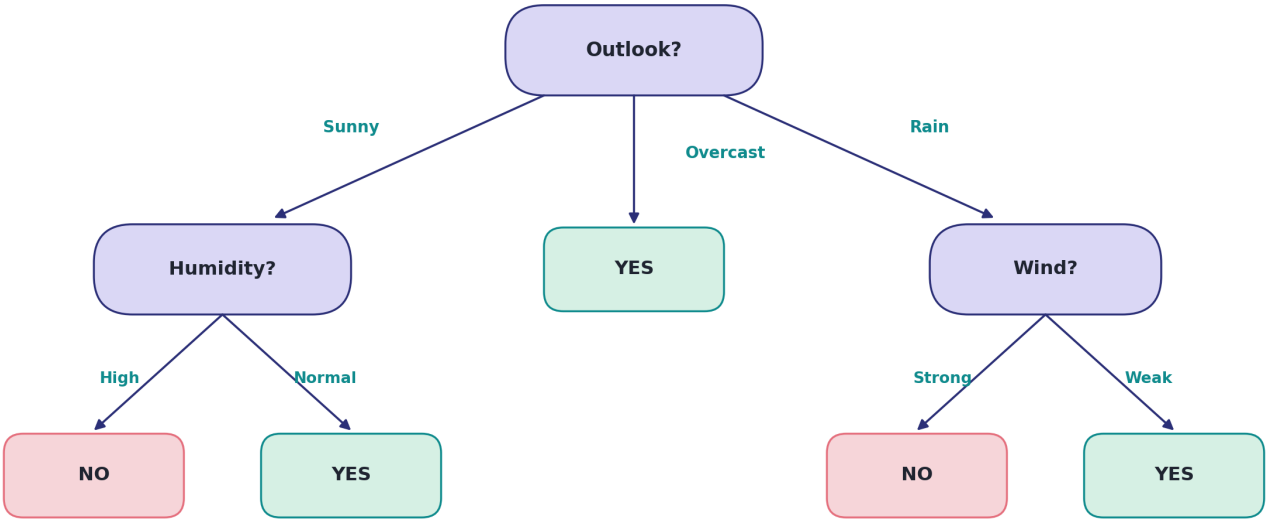


Fig 4.6 Decision tree learned for the 'Play Tennis' data

- **Overfitting control:** *pre-pruning* (stop growing early) and *post-pruning* (cut back subtrees that do not improve validation accuracy).
- **Gain ratio** (C4.5) corrects ID3's bias toward attributes with many values. **Continuous attributes** are handled by choosing a threshold split.

8. Explanation-Based Learning (EBL)

Inductive learning needs many examples. **EBL** learns from a **single** example by using **background knowledge** (a domain theory) to **explain** why the example is an instance of the goal concept, then **generalises** the explanation into a reusable rule.

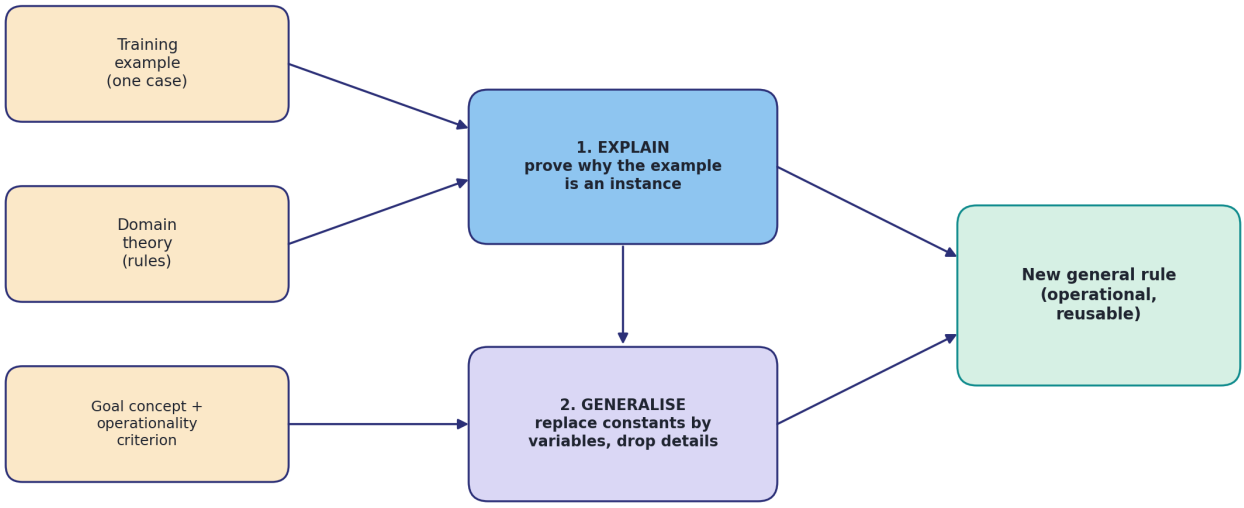


Fig 4.8 Explanation-based learning (EBL): learn a general rule from a single explained example

- **Inputs:** a goal concept, one training example, a domain theory (rules), and an **operationality criterion** (the concept must be expressed with easily checked features).
- **Step 1, Explain:** prove that the example satisfies the goal concept using the domain theory, giving a proof tree. Features not used in the proof are irrelevant.
- **Step 2, Generalise:** replace constants with variables, keeping only the conditions that appear at the leaves of the proof.

EXAMPLE
Concept: Cup. Domain theory: Liftable(x) ∧ HoldsLiquid(x) → Cup(x); Light(x) ∧ HasHandle(x) → Liftable(x); HasConcavity(x) ∧ PointsUp(x) → HoldsLiquid(x).
Example: Obj1 is red, light, has a handle, has an upward-pointing concavity, and is made of plastic. The explanation never uses *red* or *plastic*, so the learned rule is: **Light(x) ∧ HasHandle(x) ∧ HasConcavity(x) ∧ PointsUp(x) → Cup(x)**.

	Inductive learning	Explanation-based learning
Examples needed	Many	One (or few)
Knowledge used	Examples only	Examples + domain theory
Result	New knowledge (may be wrong)	A faster, more efficient form of existing knowledge (always correct if the theory is)
Typical outcome	Hypothesis / decision tree	Operational general rule

9. Statistical Learning Methods

Statistical learning treats learning as **probabilistic inference**, which is useful because data is noisy and uncertain.

Bayesian learning

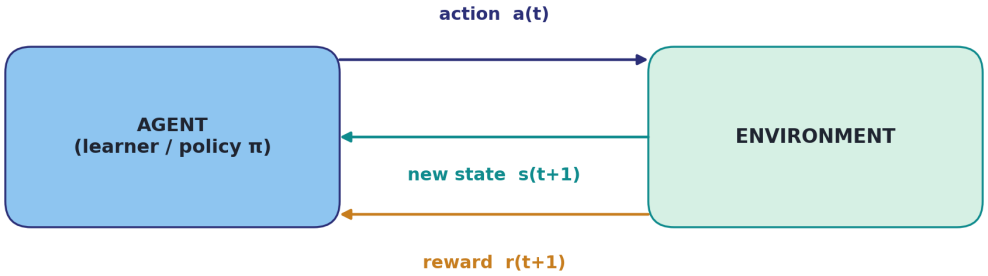
It computes the probability of each hypothesis given the data using Bayes' rule: **P(h | d) = α · P(d | h) · P(h)**. Predictions average over all hypotheses weighted by their posterior probability. Because this is often too costly, two approximations are used:

Method	Choose the hypothesis that…	Formula
MAP (maximum a posteriori)	has the highest posterior probability	h_{MAP} = argmax P(d | h) P(h)
ML (maximum likelihood)	makes the data most likely (assumes all priors are equal)	h_{ML} = argmax P(d | h)

- **Naive Bayes classifier:** assumes attributes are independent given the class, so $P(C \mid x_1 \ldots x_n) \propto P(C) \cdot \Pi\, P(x_i \mid C)$. Very fast, needs little data, used for spam filtering and text classification.
- **Instance-based learning (k-nearest neighbours):** classify a new point by a majority vote of the k closest stored examples. No training phase, but slow at prediction time.
- **Neural networks:** layers of simple units with adjustable weights, trained with back-propagation to reduce the error. They are the basis of deep learning.
- **Support vector machines and EM (clustering with hidden variables)** are other widely used statistical methods.

10. Reinforcement Learning

In **reinforcement learning (RL)** there is no teacher giving correct answers. The agent acts, then receives a numerical **reward** (or punishment), and must learn which actions lead to the highest **long-term** reward. Think of training a dog with treats.



Goal: learn a policy that maximises the total (discounted) future reward

Fig 4.9 The reinforcement-learning loop

Element	Meaning
State s	The agent's current situation
Action a	A choice available to the agent
Reward r	Immediate feedback signal after an action
Policy π(s)	The agent's strategy: which action to take in each state
Utility / value U(s)	Expected total future reward from state s (rewards are discounted by γ, 0 ≤ γ ≤ 1)

Types of reinforcement learning

- **Passive RL:** the policy is fixed; the agent only learns the utility of states by observing, e.g., *adaptive dynamic programming* and *temporal-difference (TD) learning*.
- **Active RL:** the agent must also decide what to do. It faces the **exploration vs exploitation** trade-off: use the best known action (exploit) or try something new (explore).
- **Q-learning** (model-free): learns an action-value function Q(s, a) directly, without needing a model of the environment.

FORMULA

$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \cdot \max_{a'} Q(s', a') - Q(s, a)]$ (α = learning rate, γ = discount factor)

EXAMPLE

Let Q(s, a) = 2, reward r = 1, max Q(s', a') = 5, α = 0.5, γ = 0.9. Target = 1 + 0.9 × 5 = 5.5. New Q = 2 + 0.5 × (5.5 − 2) = **3.75**. The estimate moves toward the target.

- **Applications:** game playing (TD-Gammon, AlphaGo), robot walking and control, self-driving, resource allocation and recommendation.

Quick Revision: Unit IV at a Glance

Topic	One-line recall
FOL	Objects + predicates + quantifiers; ∀ with →, ∃ with ∧
UI / EI	Universal: any ground term. Existential: a new Skolem constant
Unification	Find θ making two expressions identical; MGU is the most general; use the occur check
Forward chaining	Facts to goal (data-driven); complete for definite clauses
Backward chaining	Goal to facts (goal-driven); basis of Prolog
Resolution	CNF + negated goal; derive the empty clause for a proof
Decision tree	Choose the split with the highest information gain
EBL	Explain one example with a domain theory, then generalise
RL	Learn from rewards; Q-learning update with α and γ

Frequently asked exam questions

- Explain propositional vs first-order inference. What are UI and EI?
- Explain unification and lifting with examples. State Generalised Modus Ponens.
- Explain forward and backward chaining with the crime example, and compare them.
- Explain the resolution procedure: CNF conversion steps and a proof by refutation.
- Construct a decision tree for the given data using information gain.
- Write notes on explanation-based learning, statistical learning methods and reinforcement learning.

ARTIFICIAL INTELLIGENCE • 23A31301T

UNIT V

t

Digital Notes | Diagrams, worked examples and exam tips

SYLLABUS COVERED IN THIS UNIT

x

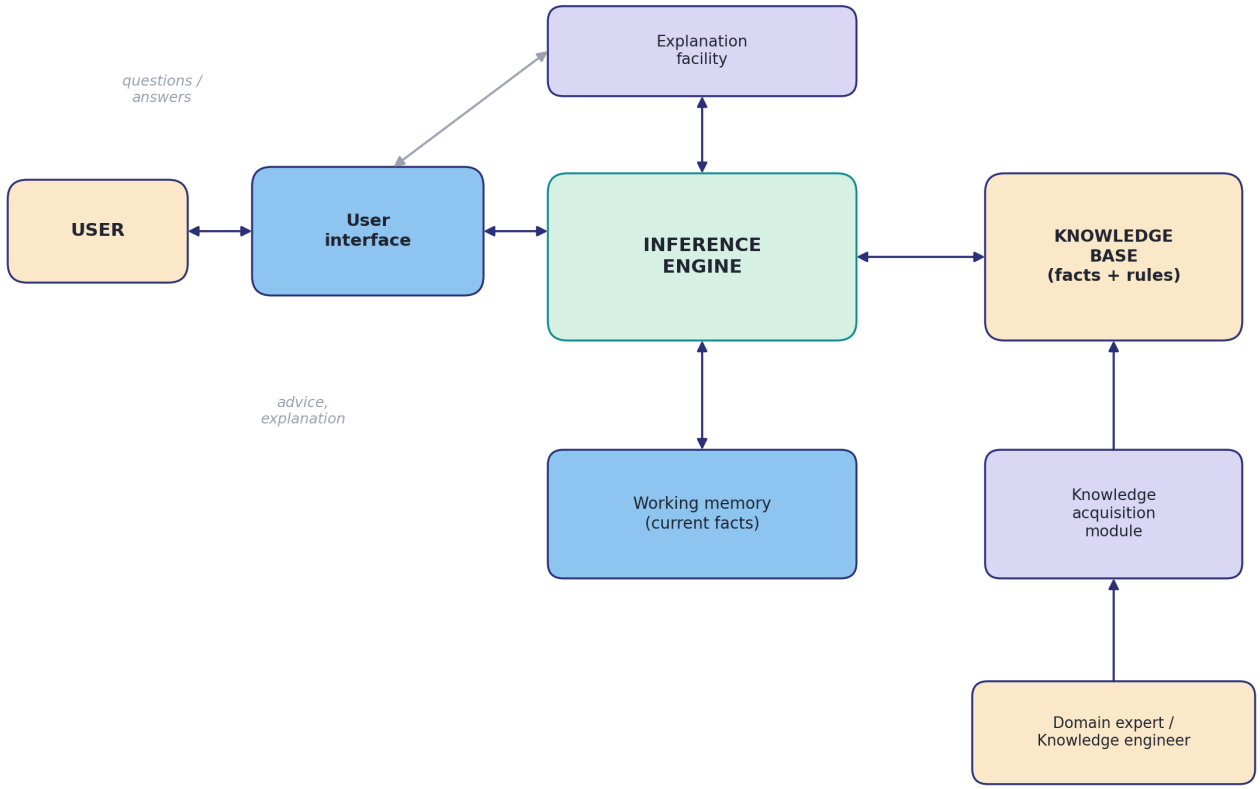


Fig 5.1 Architecture of an expert system

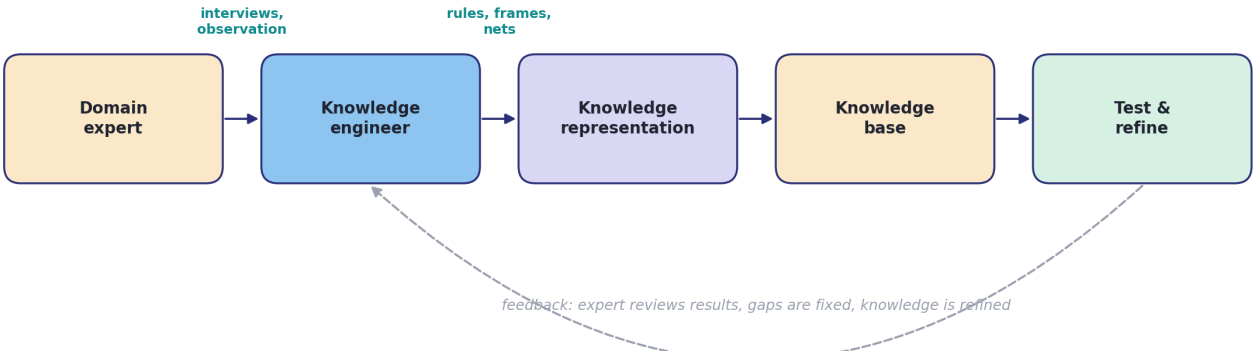


Fig 5.2 The knowledge acquisition process: from expert's head to a working knowledge base

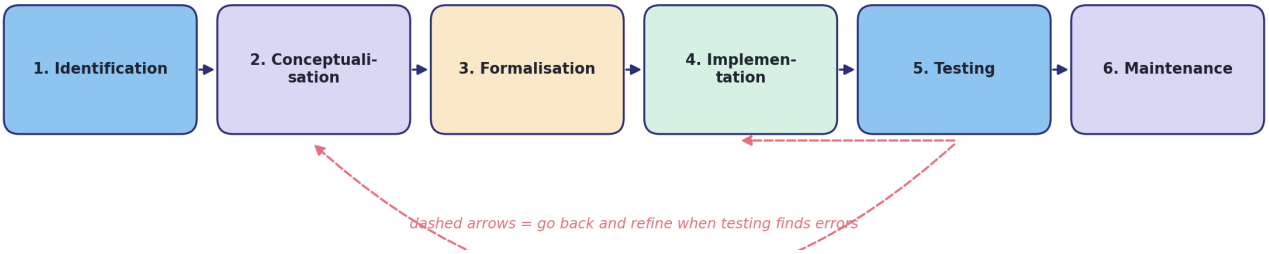


Fig 5.3 Expert system development life cycle

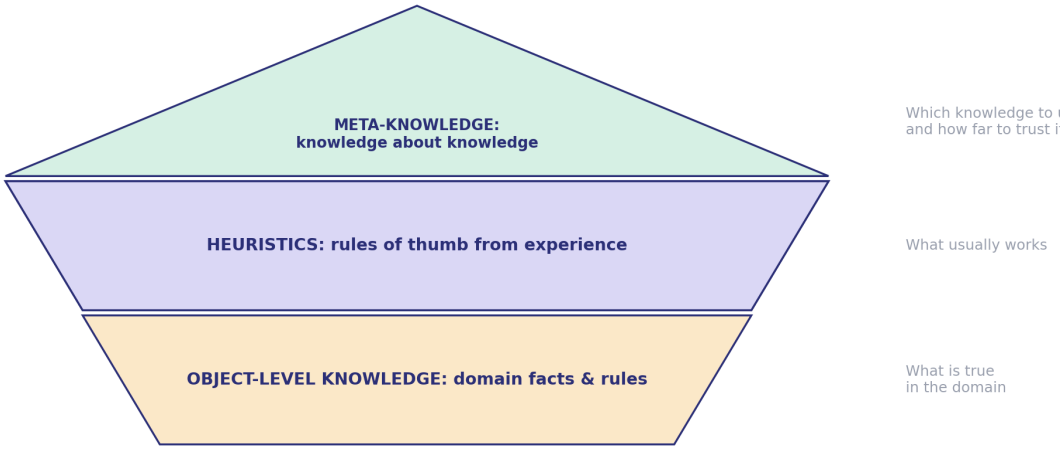


Fig 5.4 Levels of knowledge in an expert system

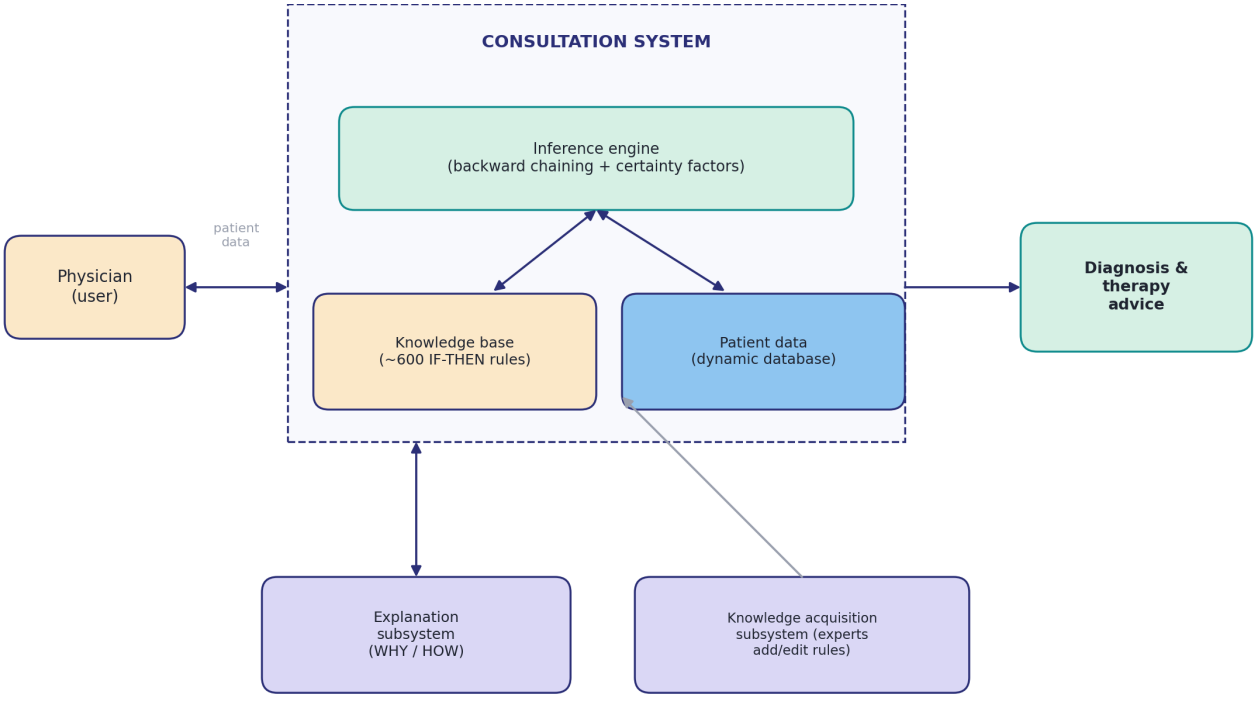


Fig 5.5 Structure of MYCIN (simplified)

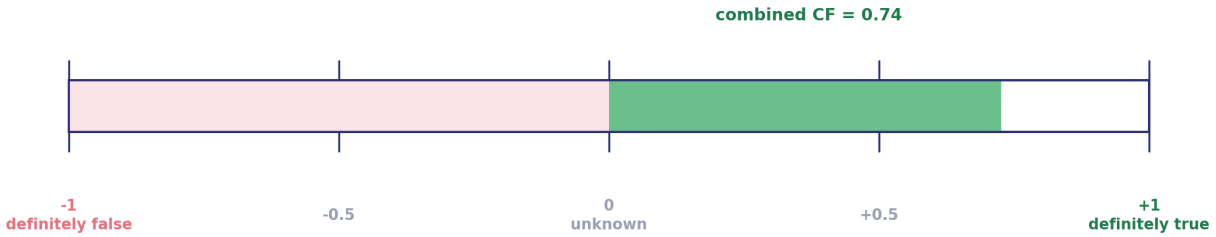


Fig 5.6 MYCIN certainty-factor scale (-1 to +1); the green bar shows the combined result of the worked example

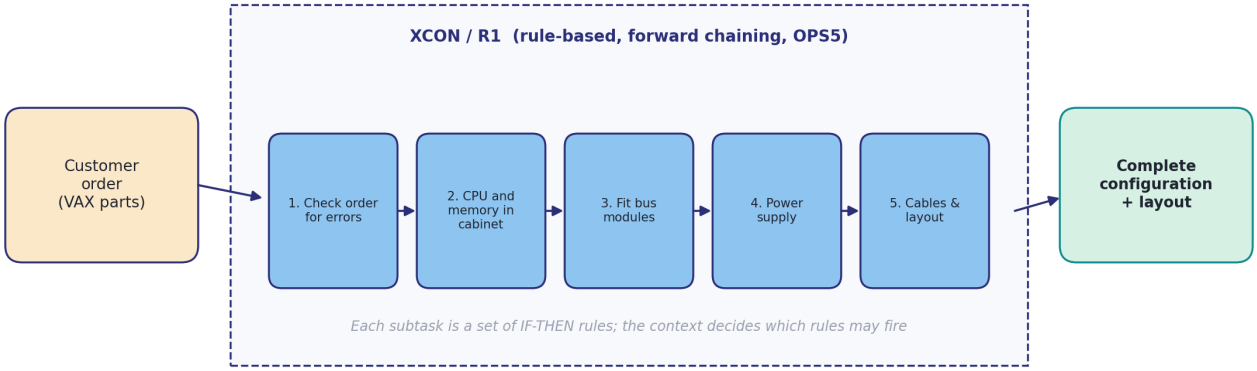


Fig 5.7 XCON configures a computer order through a chain of subtasks

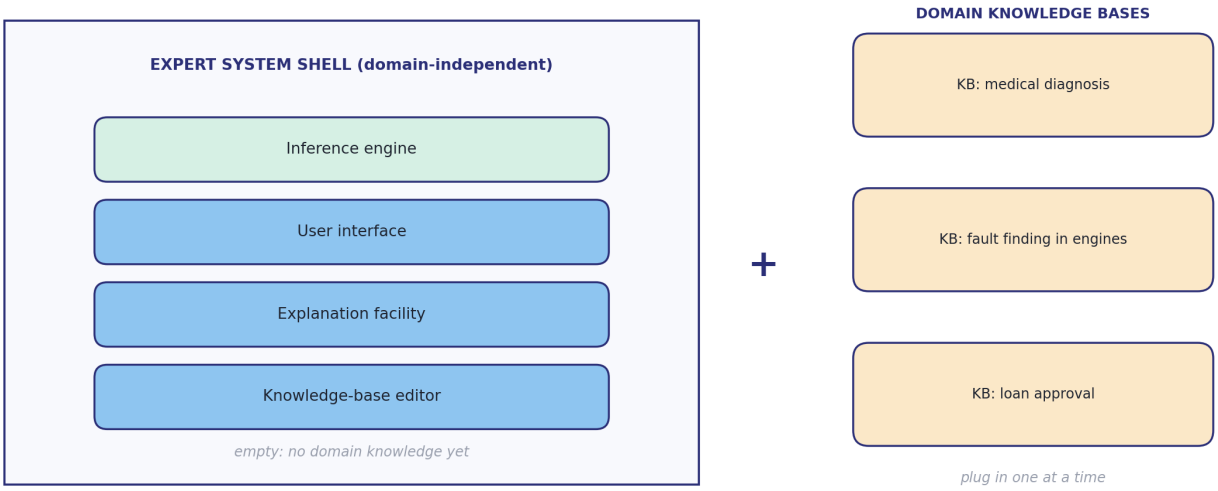


Fig 5.8 An expert system shell is an ES with its knowledge removed; fill it with a new knowledge base to build a new ES